

Áreas EDR

Table of contents

1	Áreas EDR	15
1.1	Áreas	15
1.2	Búsqueda y tags	16
1.3	Reglas de clasificación	16
1.4	Área 00 — General	16
1.5	Área 10 — Funcional	17
1.6	Área 20 — Arquitectura	17
1.7	Área 30 — Constraints	18
1.8	Área 40 — Datos	18
1.9	Área 50 — Integración	19
1.10	Área 60 — Seguridad	19
1.11	Área 70 — Calidad	20
1.12	Área 80 — Operación	20
1.13	Área 99 — Pendiente de clasificar	21
1.14	Criterio de asignación	21
1.15	Una única clasificación principal	21
1.16	El área no depende de la implementación	22
1.17	Estabilidad	22
1.18	Separación entre áreas	23
1.19	Creación de nuevas áreas	23
1.20	Principio de evolución	24
1.21	Regla fundamental	24
2	Áreas EDR	25
3	Áreas EDR	26
3.1	Áreas	26
3.2	Área 10 — Funcional	27
3.3	Área 20 — Arquitectura	28
3.4	Área 30 — Constraints	28
3.5	Área 40 — Datos	29
3.6	Área 50 — Integración	29
3.7	Área 60 — Seguridad	30
3.8	Área 70 — Calidad	30
3.9	Área 80 — Operación	31

3.10	Área 99 — Pendiente de clasificar	31
3.11	Criterio de asignación	32
3.12	Una única clasificación principal	32
3.13	El área no depende de la implementación	33
3.14	Estabilidad	33
3.15	Separación entre áreas	33
3.16	Creación de nuevas áreas	34
3.17	Principio de evolución	34
3.18	Regla fundamental	35
I	General	36
4	Registro único de decisiones de ingeniería	38
4.1	Contexto	38
4.2	Decisión	38
4.3	Alternativas consideradas	38
4.4	Consecuencias	38
5	Identificación y clasificación de los EDR	39
5.1	Contexto	39
5.2	Decisión	39
5.3	Alternativas consideradas	39
5.4	Consecuencias	39
6	Partir de la necesidad y no de las herramientas disponibles	40
6.1	Contexto	40
6.2	Decisión	40
6.3	Alternativas consideradas	40
6.4	Consecuencias	40
7	Encapsular la complejidad interna	41
7.1	Contexto	41
7.2	Decisión	41
7.3	Alternativas consideradas	41
7.4	Consecuencias	41
8	Automatizar lo repetible y mantener manual lo excepcional	42
8.1	Contexto	42
8.2	Decisión	42
8.3	Alternativas consideradas	42
8.4	Consecuencias	42

9	Limitar la velocidad del sistema a la capacidad humana sostenible	43
9.1	Contexto	43
9.2	Decisión	43
9.3	Alternativas consideradas	43
9.4	Consecuencias	43
10	Organizar la obra de IASI en tres volúmenes	44
10.1	Contexto	44
10.2	Decisión	44
10.3	Alternativas consideradas	44
10.4	Consecuencias	44
11	Separar el proceso vivido del resultado consolidado	45
11.1	Contexto	45
11.2	Decisión	45
11.3	Alternativas consideradas	45
11.4	Consecuencias	45
12	Hacer emerger las prácticas desde necesidades reales	46
12.1	Contexto	46
12.2	Decisión	46
12.3	Alternativas consideradas	46
12.4	Consecuencias	46
13	Adoptar desarrollo dirigido por especificaciones	47
13.1	Contexto	47
13.2	Decisión	47
13.3	Alternativas consideradas	47
13.4	Consecuencias	47
14	Lab execution model	48
15	Lab execution model	49
15.1	Contexto	49
15.2	Decisión	49
15.3	El lab como secuencia	50
15.4	Separación entre lab y proyecto	51
15.4.1	El lab contiene	51
15.4.2	El repositorio contiene	51
15.5	Estados oficiales del lab	51
15.6	Identidad y orden editorial	52
15.7	Historia del usuario	53
15.8	Regreso a un checkpoint	53
15.9	Reproducibilidad	54

15.10	Consecuencias	55
15.11	Principios derivados	55
16	Organizar el flujo del repositorio mediante nombres semánticos	56
17	Organizar el flujo del repositorio mediante nombres semánticos	57
17.1	Contexto	57
17.2	Decisión	57
17.3	Rationale	58
17.4	Alternativas consideradas	59
17.4.1	Mantener los prefijos A-, B-, C-, D-	59
17.4.2	Mantener engine/	59
17.4.3	Utilizar model/	59
17.4.4	Utilizar deliverables/	59
17.5	Consecuencias	59
17.6	Principios derivados	60
18	Persistir el conocimiento de ingeniería en artefactos y no en el chat	61
19	Persistir el conocimiento de ingeniería en artefactos y no en el chat	62
19.1	Contexto	62
19.2	Decisión	62
19.3	Rationale	63
19.4	Alternativas consideradas	63
19.4.1	Mantener el conocimiento principal en prompts o conversaciones	63
19.4.2	Documentar únicamente decisiones grandes	63
19.5	Consecuencias	63
20	Asociar el Sistema Inteligente al ingeniero y no al equipo	64
21	Asociar el Sistema Inteligente al ingeniero y no al equipo	65
21.1	Contexto	65
21.2	Decisión	65
21.3	Rationale	65
21.4	Alternativas consideradas	66
21.4.1	Un SI compartido por todo el equipo	66
21.4.2	Tratar el SI como un miembro autónomo del equipo	66
21.5	Consecuencias	66
22	Separar roles, personas y puestos	67
23	Separar roles, personas y puestos	68
23.1	Contexto	68
23.2	Decisión	68

23.3	Rationale	68
23.4	Alternativas consideradas	69
23.4.1	Asociar un rol a un puesto fijo	69
23.4.2	Permitir que una persona actúe simultáneamente desde varios roles sin distinguirlos	69
23.5	Consecuencias	69
24	Adoptar TOML como formato preferente de configuración	70
25	Adoptar TOML como formato preferente de configuración	71
25.1	Contexto	71
25.2	Decisión	71
25.3	Rationale	72
25.4	Alternativas consideradas	72
25.4.1	YAML como formato general	73
25.4.2	JSON como configuración	73
25.5	Consecuencias	73
II	Funcional	75
26	Usar iasi-home como puerta de entrada visual al ecosistema	77
26.1	Contexto	77
26.2	Decisión	77
26.3	Alternativas consideradas	77
26.4	Consecuencias	77
27	Distinguir libros de productos y artefactos en iasi-home	78
27.1	Contexto	78
27.2	Decisión	78
27.3	Alternativas consideradas	78
27.4	Consecuencias	78
28	Ofrecer iasi-home en castellano e inglés con detección automática	79
28.1	Contexto	79
28.2	Decisión	79
28.3	Alternativas consideradas	79
28.4	Consecuencias	79
29	Generar un catálogo por área de EDR	80
29.1	Contexto	80
29.2	Decisión	80
29.3	Alternativas consideradas	80
29.4	Consecuencias	80

30 Crear EDR mediante un Addin de RStudio	81
30.1 Contexto	81
30.2 Decisión	81
30.3 Alternativas consideradas	81
30.4 Consecuencias	81
31 Crear una presentación de introducción a IASI	82
31.1 Contexto	82
31.2 Decisión	82
31.3 Alternativas consideradas	82
31.4 Consecuencias	83
32 Proporcionar un agente IASI capaz de ejecutar trabajo sobre el workspace	84
32.1 Contexto	84
32.2 Decisión	84
32.3 Alternativas consideradas	85
32.3.1 Utilizar únicamente Ollama u Open WebUI	85
32.3.2 Dependier exclusivamente de agentes externos	86
32.3.3 Construir inmediatamente un agente completo desde cero	86
32.4 Consecuencias	86
33 Distinguir productos y subproductos en iasi-home	87
34 Distinguir productos y subproductos en iasi-home	88
34.1 Contexto	88
34.2 Decisión	88
34.3 Rationale	88
34.4 Alternativas consideradas	89
34.4.1 Mantener Artefactos como categoría principal	89
34.4.2 Considerar producto todo lo que IASI haya creado	89
34.5 Consecuencias	89
35 Congelar iasi-home por cobertura y navegación, no por completitud de contenido	90
36 Congelar iasi-home por cobertura y navegación, no por completitud de contenido	91
36.1 Contexto	91
36.2 Decisión	91
36.3 Rationale	92
36.4 Alternativas consideradas	92
36.4.1 Esperar a terminar cada elemento antes de mostrarlo	92
36.4.2 Mostrar únicamente las piezas terminadas	92
36.5 Consecuencias	92

III Arquitectura	93
37 Mantener iasi-edr como repositorio independiente	95
37.1 Contexto	95
37.2 Decisión	95
37.3 Alternativas consideradas	95
37.4 Consecuencias	95
38 Publicar iasi-edr como Quarto book estructurado	96
38.1 Contexto	96
38.2 Decisión	96
38.3 Alternativas consideradas	96
38.4 Consecuencias	96
39 Definir las áreas EDR en _iasi.yml	97
39.1 Contexto	97
39.2 Decisión	97
39.3 Alternativas consideradas	97
39.4 Consecuencias	97
40 Organizar físicamente los EDR por área	98
40.1 Contexto	98
40.2 Decisión	98
40.3 Alternativas consideradas	98
40.4 Consecuencias	98
41 Regenerar los índices EDR en pre-render	99
41.1 Contexto	99
41.2 Decisión	99
41.3 Alternativas consideradas	99
41.4 Consecuencias	99
42 Mantener la lógica específica de EDR fuera de iasi.quarto	100
42.1 Contexto	100
42.2 Decisión	100
42.3 Alternativas consideradas	100
42.4 Consecuencias	100
43 Separar la configuración común de los perfiles HTML y PDF	101
43.1 Contexto	101
43.2 Decisión	101
43.3 Alternativas consideradas	101
43.4 Consecuencias	101

44 Usar la navbar como navegación principal de iasi-edr en HTML	102
44.1 Contexto	102
44.2 Decisión	102
44.3 Alternativas consideradas	102
44.4 Consecuencias	102
45 Componer el estilo HTML con SCSS común y específico	103
45.1 Contexto	103
45.2 Decisión	103
45.3 Alternativas consideradas	103
45.4 Consecuencias	103
46 Mantener iasi-graphics deliberadamente simple	104
46.1 Contexto	104
46.2 Decisión	104
46.3 Alternativas consideradas	104
46.4 Consecuencias	104
47 Crear un espacio dedicado para capacidades agentic de IASI	105
47.1 Contexto	105
47.2 Decisión	105
47.3 Alternativas consideradas	105
47.4 Consecuencias	105
48 Separar el entorno de laboratorios mediante una máquina virtual dedicada	106
48.1 Contexto	106
48.2 Decisión	106
48.3 Alternativas consideradas	106
48.4 Consecuencias	107
49 Usar HTMLSlides para materializar Introducing IASI	108
49.1 Contexto	108
49.2 Decisión	109
49.3 Alternativas consideradas	109
49.3.1 PowerPoint	109
49.3.2 Gamma	110
49.3.3 Prezi	110
49.3.4 HTML desarrollado específicamente para la presentación	111
49.3.5 HTMLSlides	111
49.4 Consecuencias	112
50 Mantener iasi-agent como proyecto separado de IASI	113
50.1 Contexto	113
50.2 Decisión	113

50.3	Alternativas consideradas	114
50.3.1	Incorporar el agente directamente dentro de IASI	114
50.3.2	Tratar el agente como parte de <code>iasi-dev</code>	114
50.3.3	Utilizar únicamente adaptadores para agentes externos	114
50.4	Consecuencias	114
51	Usar OpenCode como base inicial de <code>iasi-agent</code>	116
51.1	Contexto	116
51.2	Decisión	116
51.3	Alternativas consideradas	117
51.3.1	Ollama + Open WebUI	117
51.3.2	Continue	118
51.3.3	Aider	118
51.3.4	Codex CLI	119
51.3.5	Construir un agente desde cero	119
51.3.6	OpenCode	120
51.4	Consecuencias	121
52	Declarar cada proyecto IASI mediante <code>_iasi.yml</code>	122
53	Declarar cada proyecto IASI mediante <code>_iasi.yml</code>	123
53.1	Contexto	123
53.2	Decisión	123
53.3	Rationale	124
53.4	Alternativas consideradas	124
53.4.1	Utilizar <code>.iasi/</code> como marcador de proyecto	124
53.4.2	Inferir que existe un proyecto por la presencia de archivos de una tecnología concreta	124
53.5	Consecuencias	124
54	Separar proyecto IASI, repositorio Git y release	125
55	Separar proyecto IASI, repositorio Git y release	126
55.1	Contexto	126
55.2	Decisión	126
55.3	Configuración	127
55.4	Rationale	127
55.5	Alternativas consideradas	127
55.5.1	Hacer que cada proyecto hijo conozca su repositorio	127
55.5.2	Tratar release como operación puramente local del proyecto	127
55.6	Consecuencias	128
56	Usar Ubuntu Server 24.04 LTS en <code>iasi-tools</code>	129

57 Usar Ubuntu Server 24.04 LTS en iasi-tools	130
57.1 Contexto	130
57.2 Decisión	130
57.3 Rationale	130
57.4 Alternativas consideradas	131
57.4.1 Debian Server	131
57.4.2 Ubuntu Server 26.04 LTS	131
57.4.3 Utilizar iasi-dev para alojar las herramientas	131
57.5 Consecuencias	131
58 Usar Ubuntu Server 24.04 LTS en iasi-ai	133
59 Usar Ubuntu Server 24.04 LTS en iasi-ai	134
59.1 Contexto	134
59.2 Decisión	134
59.3 Rationale	134
59.4 Alternativas consideradas	135
59.4.1 Debian Server	135
59.4.2 Ubuntu Server 26.04 LTS	135
59.4.3 Distribución especializada para IA o GPU	135
59.4.4 Ejecutar el runtime de IA en iasi-tools	135
59.5 Consecuencias	136
IV Constraints	137
60 Diseñar las publicaciones IASI para escritorio	139
60.1 Contexto	139
60.2 Decisión	139
60.3 Alternativas consideradas	139
60.4 Consecuencias	139
61 Evitar la generalización prematura	140
61.1 Contexto	140
61.2 Decisión	140
61.3 Alternativas consideradas	140
61.4 Consecuencias	140
62 Construir manualmente la infraestructura antes de automatizarla	141
62.1 Contexto	141
62.2 Decisión	141
62.3 Alternativas consideradas	141
62.4 Consecuencias	141

63 Usar Windows como sistema operativo de referencia para los labs	142
63.1 Contexto	142
63.2 Decisión	142
63.3 Alternativas consideradas	142
63.4 Consecuencias	143
 V Datos	 144
64 Usar el front matter como fuente de metadatos del EDR	146
64.1 Contexto	146
64.2 Decisión	146
64.3 Alternativas consideradas	146
64.4 Consecuencias	146
65 Tratar los índices de área como datos derivados	147
65.1 Contexto	147
65.2 Decisión	147
65.3 Alternativas consideradas	147
65.4 Consecuencias	147
66 Hacer los EDR consultables por metadatos y texto completo	148
66.1 Contexto	148
66.2 Decisión	148
66.3 Alternativas consideradas	148
66.4 Consecuencias	148
 VI Integración	 149
67 Distinguir MCP del servidor MCP	151
67.1 Contexto	151
67.2 Decisión	151
67.3 Alternativas consideradas	151
67.4 Consecuencias	151
68 Exponer PlantUML como capacidad utilizable por modelos	152
68.1 Contexto	152
68.2 Decisión	152
68.3 Alternativas consideradas	152
68.4 Consecuencias	152
69 Clasificar los productos técnicos por su forma de uso	153
69.1 Contexto	153

69.2 Decisión	153
69.3 Alternativas consideradas	153
69.4 Consecuencias	153
VII Seguridad	154
VIII Calidad	156
70 Validar la estructura Cuarto antes de preparar o renderizar	158
70.1 Contexto	158
70.2 Decisión	158
70.3 Alternativas consideradas	158
70.4 Consecuencias	158
71 Fallar ante inconsistencias estructurales del registro EDR	159
71.1 Contexto	159
71.2 Decisión	159
71.3 Alternativas consideradas	159
71.4 Consecuencias	159
72 Regenerar de forma determinista los artefactos derivados	160
72.1 Contexto	160
72.2 Decisión	160
72.3 Alternativas consideradas	160
72.4 Consecuencias	160
73 Preservar el formato deliberado del código Go	161
73.1 Contexto	161
73.2 Decisión	161
73.3 Rationale	162
73.4 Alternativas consideradas	162
73.4.1 Mantener <code>gofmt</code> como obligatorio	162
73.4.2 Utilizar <code>gofmt</code> por defecto y permitir excepciones puntuales	162
73.4.3 Desactivar únicamente el formateo al guardar	162
73.5 Consecuencias	163
73.6 Principios derivados	163
74 Separar contenido derivable y contenido de autor en artefactos regenerables	164
75 Separar contenido derivable y contenido de autor en artefactos regenerables	165
75.1 Contexto	165
75.2 Decisión	165

75.3	Rationale	165
75.4	Alternativas consideradas	166
75.4.1	Regenerar siempre el documento completo	166
75.4.2	Dejar de regenerar cuando exista contenido manual	166
75.4.3	Modificar automáticamente el contenido de autor cuando cambien sus dependencias	166
75.5	Consecuencias	166
IX	Operación	167
76	Usar HTML como perfil por defecto	169
76.1	Contexto	169
76.2	Decisión	169
76.3	Alternativas consideradas	169
76.4	Consecuencias	169
77	Separar los directorios de salida por perfil	170
77.1	Contexto	170
77.2	Decisión	170
77.3	Alternativas consideradas	170
77.4	Consecuencias	170
78	Usar scripts R reutilizables para preview y render	171
78.1	Contexto	171
78.2	Decisión	171
78.3	Alternativas consideradas	171
78.4	Consecuencias	171
79	Separar herramientas principales de trabajo y laboratorios locales	172
79.1	Contexto	172
79.2	Decisión	172
79.3	Alternativas consideradas	172
79.4	Consecuencias	172
80	Postprocesar la salida de Quarto sin forzar su comportamiento	173
80.1	Contexto	173
80.2	Decisión	173
80.3	Alternativas consideradas	174
80.4	Consecuencias	174
X	Pendiente de clasificar	175

1 Áreas EDR

Los Engineering Decision Records de IASI utilizan identificadores con la forma:

EDR-**nnmmm**

donde:

- **nn** identifica el **área principal de ingeniería** de la decisión.
- **mmm** identifica secuencialmente la decisión dentro de esa área.

El área permite conocer la naturaleza principal de una decisión antes incluso de abrir el EDR.

1.1 Áreas

nn	Área	Criterio
00	General	Decisiones transversales o relativas al propio funcionamiento de IASI.
10	Funcional	Qué debe hacer el sistema y qué comportamiento debe ofrecer.
20	Arquitectura	Estructura del sistema, componentes, responsabilidades y relaciones.
30	Constraints	Restricciones que condicionan o limitan las posibles soluciones.
40	Datos	Modelos, persistencia, intercambio, ciclo de vida y gobierno del dato.
50	Integración	Interfaces, protocolos y comunicación entre sistemas o componentes.
60	Seguridad	Identidad, acceso, protección, amenazas y controles.
70	Calidad	Validación, pruebas, observabilidad y atributos de calidad.
80	Operación	Despliegue, ejecución, configuración, infraestructura y explotación.
99	Pendiente de clasificar	Decisiones registradas provisionalmente cuya clasificación definitiva todavía no se ha determinado.

1.2 Búsqueda y tags

En la publicación HTML, la lupa realiza una búsqueda de texto completo sobre el libro.

Cada EDR pertenece a una única **área principal**, pero puede incorporar varios **tags** para describir temas transversales. Los tags se definen una sola vez en el front matter y se muestran automáticamente en la página del EDR, por lo que también forman parte de la búsqueda.

La regla es sencilla:

- el **área** responde a dónde se clasifica principalmente una decisión;
- los **tags** responden a de qué habla la decisión y permiten encontrarla desde otros ángulos.

1.3 Reglas de clasificación

Cada EDR pertenece a **una única área principal**.

El área representa la naturaleza de la decisión, no:

- la herramienta utilizada;
- la tecnología elegida;
- el repositorio donde se implementa;
- el equipo responsable.

Cuando una decisión afecta a varios ámbitos, se asigna al área que mejor represente **el objeto principal de la decisión**.

Los efectos secundarios pueden expresarse mediante los metadatos del EDR.

1.4 Área 00 — General

00 está reservado para decisiones generales o transversales.

También pertenece a esta área cualquier decisión relacionada con el propio sistema de Engineering Decision Records.

Por ejemplo:

EDR-00001

puede definir el modelo, estructura y reglas de los propios EDR.

00 no debe utilizarse simplemente porque una decisión resulte difícil de clasificar.

1.5 Área 10 — Funcional

El área 10 agrupa decisiones relacionadas con **qué debe hacer el sistema**.

Incluye decisiones sobre:

- comportamiento funcional;
- capacidades ofrecidas;
- reglas de negocio;
- interacción entre el sistema y sus usuarios;
- resultados esperados;
- criterios funcionales.

Una decisión pertenece a esta área cuando su cuestión principal puede formularse aproximadamente como:

¿Qué comportamiento debe ofrecer el sistema?

1.6 Área 20 — Arquitectura

El área 20 agrupa decisiones relacionadas con **cómo se estructura el sistema**.

Incluye decisiones sobre:

- componentes;
- responsabilidades;
- relaciones entre componentes;
- separación de responsabilidades;
- organización interna;
- estilos y patrones arquitectónicos;
- distribución de capacidades;
- fronteras entre subsistemas.

Una decisión pertenece a esta área cuando su cuestión principal puede formularse aproximadamente como:

¿Cómo organizamos el sistema para satisfacer sus necesidades?

1.7 Área 30 — Constraints

El área 30 agrupa decisiones derivadas de **restricciones que condicionan el espacio de soluciones posibles**.

Incluye restricciones:

- técnicas;
- organizativas;
- operativas;
- regulatorias;
- económicas;
- temporales;
- de compatibilidad;
- de plataforma;
- impuestas por sistemas externos.

Una decisión pertenece a esta área cuando su cuestión principal parte de una condición que no puede ignorarse libremente.

Los constraints no describen necesariamente qué queremos hacer, sino **qué límites debe respetar cualquier solución válida**.

1.8 Área 40 — Datos

El área 40 agrupa decisiones relacionadas con la **representación, persistencia, intercambio y ciclo de vida de los datos**.

Incluye decisiones sobre:

- modelos de datos;
- estructuras y formatos;
- almacenamiento y persistencia;
- migraciones;
- retención y ciclo de vida;
- calidad y gobierno del dato;
- intercambio y transformación de información.

Una decisión pertenece a esta área cuando el elemento principal sobre el que se decide es el dato y su tratamiento.

1.9 Área 50 — Integración

El área 50 agrupa decisiones relacionadas con la **comunicación entre sistemas, servicios o componentes**.

Incluye decisiones sobre:

- interfaces;
- APIs;
- protocolos;
- mensajería;
- contratos de integración;
- interoperabilidad;
- mecanismos de comunicación;
- dependencias con sistemas externos.

Una decisión pertenece a esta área cuando su cuestión principal es cómo se relacionan o intercambian información distintos elementos del sistema.

1.10 Área 60 — Seguridad

El área 60 agrupa decisiones relacionadas con la **protección del sistema, sus usuarios y su información**.

Incluye decisiones sobre:

- identidad;
- autenticación;
- autorización;
- gestión de secretos;
- protección de datos;
- amenazas;
- controles;
- aislamiento;
- trazabilidad de seguridad.

Una decisión pertenece a esta área cuando su propósito principal es reducir riesgos o establecer mecanismos de protección.

1.11 Área 70 — Calidad

El área 70 agrupa decisiones relacionadas con la **validación del sistema y sus atributos de calidad**.

Incluye decisiones sobre:

- pruebas;
- acceptance tests;
- validación;
- verificación;
- observabilidad;
- métricas;
- rendimiento;
- fiabilidad;
- mantenibilidad;
- criterios de calidad.

Una decisión pertenece a esta área cuando define cómo demostrar, medir o asegurar que el sistema cumple las propiedades esperadas.

1.12 Área 80 — Operación

El área 80 agrupa decisiones relacionadas con la **ejecución y explotación del sistema**.

Incluye decisiones sobre:

- despliegue;
- configuración;
- infraestructura;
- entornos;
- ejecución;
- automatización operativa;
- monitorización;
- recuperación;
- mantenimiento en producción.

Una decisión pertenece a esta área cuando su cuestión principal es cómo ejecutar, desplegar, configurar o mantener el sistema en funcionamiento.

1.13 Área 99 — Pendiente de clasificar

99 está reservado para decisiones que deben registrarse, pero cuya clasificación definitiva todavía no está clara.

Su uso es **provisional**.

Debe utilizarse para evitar dos problemas:

- retrasar el registro de una decisión porque todavía no se sabe dónde clasificarla;
- forzar una clasificación prematura que después resulte incorrecta.

99 no es una categoría residual ni un destino permanente.

Cuando se determine el área correcta, la decisión debe trasladarse a su clasificación definitiva.

1.14 Criterio de asignación

Para determinar **nn**, debe identificarse primero la naturaleza principal de la decisión.

Puede utilizarse esta secuencia:

1. ¿Cuál es el problema que estamos resolviendo?
2. ¿Qué aspecto de la ingeniería estamos decidiendo?
3. ¿Dónde tendría sentido buscar esta decisión dentro de varios años?
4. ¿Seguiría perteneciendo a la misma área si cambiase la tecnología utilizada?

La respuesta determina el área principal.

En caso de duda entre varias áreas, debe seleccionarse aquella que mejor represente **la razón por la que existe la decisión**.

Si todavía no existe información suficiente para hacerlo con criterio, se utiliza provisionalmente el área 99.

1.15 Una única clasificación principal

Una decisión puede afectar simultáneamente a múltiples ámbitos.

Por ejemplo, una decisión arquitectónica puede tener consecuencias funcionales, operativas o de seguridad.

Esto no implica que deba pertenecer a varias áreas.

Cada EDR tiene un único **nn**.

Los demás efectos se expresan mediante metadatos como:

```
tags:
  - publication
  - validation

affects:
  - iasi-quarto
  - iasi-book-I

related:
  - EDR-20002
```

El identificador proporciona la clasificación principal.

Los metadatos proporcionan el contexto adicional.

1.16 El área no depende de la implementación

nn no debe asignarse según:

- el lenguaje de programación;
- el framework;
- la herramienta;
- el producto;
- el repositorio;
- el proveedor;
- el agente de IA que implemente la decisión.

Por ejemplo, una decisión sobre cómo se estructura la publicación de IASI puede implementarse inicialmente en `iasi-quarto`.

Eso no convierte automáticamente la decisión en una decisión sobre Quarto.

La pregunta relevante es qué se está decidiendo desde el punto de vista de ingeniería.

1.17 Estabilidad

Una vez asignado un código nn a un área:

- su significado no cambia;
- no se reutiliza para otro ámbito;

- los EDR clasificados conservan su identificador.

Los códigos de área son permanentes.

El área 99 constituye la excepción: representa explícitamente un estado provisional pendiente de clasificación.

1.18 Separación entre áreas

Los códigos se asignan con separación deliberada:

```
00
10
20
30
40
50
60
70
80
99
```

Esto mantiene una clasificación legible y deja espacio para incorporar nuevas áreas si la evolución de IASI lo hace necesario.

1.19 Creación de nuevas áreas

La lista de áreas no pretende definir anticipadamente todos los ámbitos posibles de la ingeniería.

Se crearán nuevas áreas cuando las decisiones reales de IASI hagan necesaria una nueva categoría estable.

Antes de crear una nueva área debe comprobarse que:

1. ninguna de las áreas existentes representa adecuadamente la decisión;
2. el nuevo ámbito tiene significado propio desde el punto de vista de ingeniería;
3. no representa simplemente una herramienta, tecnología o producto;
4. previsiblemente podrá contener múltiples decisiones;
5. su significado puede mantenerse estable en el tiempo.

La taxonomía debe evolucionar a partir de necesidades reales.

1.20 Principio de evolución

IASI no pretende construir anticipadamente una clasificación completa de todas las decisiones de ingeniería posibles.

Las áreas se incorporarán cuando aparezcan decisiones reales que justifiquen su existencia.

Las áreas no se inventan por anticipado. Se descubren cuando las decisiones de ingeniería las hacen necesarias.

1.21 Regla fundamental

nn clasifica la naturaleza principal de la decisión. Los metadatos describen todo lo demás.

El identificador debe permitir saber dónde buscar una decisión.

El contenido y los metadatos explican su alcance, sus relaciones y sus consecuencias.

2 Áreas EDR

3 Áreas EDR

Los Engineering Decision Records de IASI utilizan identificadores con la forma:

```
EDR-nnmmm
```

donde:

- **nn** identifica el **área principal de ingeniería** de la decisión.
- **mmm** identifica secuencialmente la decisión dentro de esa área.

El área permite conocer la naturaleza principal de una decisión antes incluso de abrir el EDR.

3.1 Áreas

O, si quieres la cabecera compacta, solo para echo:

```
::: {.cell tbl-colwidths='[10,20,70]'}  
::: {.cell-output-display}
```

nn	Área	Criterio
00	General	Decisiones transversales o relativas al propio funcionamiento
10	Funcional	Qué debe hacer el sistema y qué comportamiento debe ofrecer.
20	Arquitectura	Estructura del sistema, componentes, responsabilidades y relaciones
30	Constraints	Restricciones que condicionan o limitan las posibles soluciones
40	Datos	Modelos, persistencia, intercambio, ciclo de vida y gobierno de datos
50	Integración	Interfaces, protocolos y comunicación entre sistemas o componentes
60	Seguridad	Identidad, acceso, protección, amenazas y controles.
70	Calidad	Validación, pruebas, observabilidad y atributos de calidad.
80	Operación	Despliegue, ejecución, configuración, infraestructura y explotación
99	Pendiente de clasificar	Decisiones registradas provisionalmente cuya clasificación definitiva

```

:::
:::

## Reglas de clasificación

Cada EDR pertenece a una única área principal.

El área representa la naturaleza de la decisión, no:

- la herramienta utilizada;
- la tecnología elegida;
- el repositorio donde se implementa;
- el equipo responsable.

Cuando una decisión afecta a varios ámbitos, se asigna al área que mejor represente el objeto.

Los efectos secundarios pueden expresarse mediante los metadatos del EDR.

## Área `00` - General

`00` está reservado para decisiones generales o transversales.

También pertenece a esta área cualquier decisión relacionada con el propio sistema de Ingeniería.

Por ejemplo:

``` text
EDR-00001

```

puede definir el modelo, estructura y reglas de los propios EDR.

00 no debe utilizarse simplemente porque una decisión resulte difícil de clasificar.

## 3.2 Área 10 — Funcional

El área 10 agrupa decisiones relacionadas con **qué debe hacer el sistema**.

Incluye decisiones sobre:

- comportamiento funcional;
- capacidades ofrecidas;

- reglas de negocio;
- interacción entre el sistema y sus usuarios;
- resultados esperados;
- criterios funcionales.

Una decisión pertenece a esta área cuando su cuestión principal puede formularse aproximadamente como:

¿Qué comportamiento debe ofrecer el sistema?

### 3.3 Área 20 — Arquitectura

El área 20 agrupa decisiones relacionadas con **cómo se estructura el sistema**.

Incluye decisiones sobre:

- componentes;
- responsabilidades;
- relaciones entre componentes;
- separación de responsabilidades;
- organización interna;
- estilos y patrones arquitectónicos;
- distribución de capacidades;
- fronteras entre subsistemas.

Una decisión pertenece a esta área cuando su cuestión principal puede formularse aproximadamente como:

¿Cómo organizamos el sistema para satisfacer sus necesidades?

### 3.4 Área 30 — Constraints

El área 30 agrupa decisiones derivadas de **restricciones que condicionan el espacio de soluciones posibles**.

Incluye restricciones:

- técnicas;
- organizativas;
- operativas;
- regulatorias;
- económicas;
- temporales;

- de compatibilidad;
- de plataforma;
- impuestas por sistemas externos.

Una decisión pertenece a esta área cuando su cuestión principal parte de una condición que no puede ignorarse libremente.

Los constraints no describen necesariamente qué queremos hacer, sino **qué límites debe respetar cualquier solución válida**.

### 3.5 Área 40 — Datos

El área 40 agrupa decisiones relacionadas con la **representación, persistencia, intercambio y ciclo de vida de los datos**.

Incluye decisiones sobre:

- modelos de datos;
- estructuras y formatos;
- almacenamiento y persistencia;
- migraciones;
- retención y ciclo de vida;
- calidad y gobierno del dato;
- intercambio y transformación de información.

Una decisión pertenece a esta área cuando el elemento principal sobre el que se decide es el dato y su tratamiento.

### 3.6 Área 50 — Integración

El área 50 agrupa decisiones relacionadas con la **comunicación entre sistemas, servicios o componentes**.

Incluye decisiones sobre:

- interfaces;
- APIs;
- protocolos;
- mensajería;
- contratos de integración;
- interoperabilidad;
- mecanismos de comunicación;
- dependencias con sistemas externos.

Una decisión pertenece a esta área cuando su cuestión principal es cómo se relacionan o intercambian información distintos elementos del sistema.

### 3.7 Área 60 — Seguridad

El área 60 agrupa decisiones relacionadas con la **protección del sistema, sus usuarios y su información**.

Incluye decisiones sobre:

- identidad;
- autenticación;
- autorización;
- gestión de secretos;
- protección de datos;
- amenazas;
- controles;
- aislamiento;
- trazabilidad de seguridad.

Una decisión pertenece a esta área cuando su propósito principal es reducir riesgos o establecer mecanismos de protección.

### 3.8 Área 70 — Calidad

El área 70 agrupa decisiones relacionadas con la **validación del sistema y sus atributos de calidad**.

Incluye decisiones sobre:

- pruebas;
- acceptance tests;
- validación;
- verificación;
- observabilidad;
- métricas;
- rendimiento;
- fiabilidad;
- mantenibilidad;
- criterios de calidad.

Una decisión pertenece a esta área cuando define cómo demostrar, medir o asegurar que el sistema cumple las propiedades esperadas.

### 3.9 Área 80 — Operación

El área 80 agrupa decisiones relacionadas con la **ejecución y explotación del sistema**.

Incluye decisiones sobre:

- despliegue;
- configuración;
- infraestructura;
- entornos;
- ejecución;
- automatización operativa;
- monitorización;
- recuperación;
- mantenimiento en producción.

Una decisión pertenece a esta área cuando su cuestión principal es cómo ejecutar, desplegar, configurar o mantener el sistema en funcionamiento.

### 3.10 Área 99 — Pendiente de clasificar

99 está reservado para decisiones que deben registrarse, pero cuya clasificación definitiva todavía no está clara.

Su uso es **provisional**.

Debe utilizarse para evitar dos problemas:

- retrasar el registro de una decisión porque todavía no se sabe dónde clasificarla;
- forzar una clasificación prematura que después resulte incorrecta.

99 no es una categoría residual ni un destino permanente.

Cuando se determine el área correcta, la decisión debe trasladarse a su clasificación definitiva.

### 3.11 Criterio de asignación

Para determinar **nn**, debe identificarse primero la naturaleza principal de la decisión.

Puede utilizarse esta secuencia:

1. ¿Cuál es el problema que estamos resolviendo?
2. ¿Qué aspecto de la ingeniería estamos decidiendo?
3. ¿Dónde tendría sentido buscar esta decisión dentro de varios años?
4. ¿Seguiría perteneciendo a la misma área si cambiase la tecnología utilizada?

La respuesta determina el área principal.

En caso de duda entre varias áreas, debe seleccionarse aquella que mejor represente **la razón por la que existe la decisión**.

Si todavía no existe información suficiente para hacerlo con criterio, se utiliza provisionalmente el área 99.

### 3.12 Una única clasificación principal

Una decisión puede afectar simultáneamente a múltiples ámbitos.

Por ejemplo, una decisión arquitectónica puede tener consecuencias funcionales, operativas o de seguridad.

Esto no implica que deba pertenecer a varias áreas.

Cada EDR tiene un único **nn**.

Los demás efectos se expresan mediante metadatos como:

```
tags:
 - publication
 - validation

affects:
 - iasi-quarto
 - iasi-book-I

related:
 - EDR-20002
```

El identificador proporciona la clasificación principal.

Los metadatos proporcionan el contexto adicional.

### 3.13 El área no depende de la implementación

`nn` no debe asignarse según:

- el lenguaje de programación;
- el framework;
- la herramienta;
- el producto;
- el repositorio;
- el proveedor;
- el agente de IA que implemente la decisión.

Por ejemplo, una decisión sobre cómo se estructura la publicación de IASI puede implementarse inicialmente en `iasi-quarto`.

Eso no convierte automáticamente la decisión en una decisión sobre Quarto.

La pregunta relevante es qué se está decidiendo desde el punto de vista de ingeniería.

### 3.14 Estabilidad

Una vez asignado un código `nn` a un área:

- su significado no cambia;
- no se reutiliza para otro ámbito;
- los EDR clasificados conservan su identificador.

Los códigos de área son permanentes.

El área 99 constituye la excepción: representa explícitamente un estado provisional pendiente de clasificación.

### 3.15 Separación entre áreas

Los códigos se asignan con separación deliberada:

```
00
10
20
30
40
50
```

60  
70  
80  
99

Esto mantiene una clasificación legible y deja espacio para incorporar nuevas áreas si la evolución de IASI lo hace necesario.

### 3.16 Creación de nuevas áreas

La lista de áreas no pretende definir anticipadamente todos los ámbitos posibles de la ingeniería.

Se crearán nuevas áreas cuando las decisiones reales de IASI hagan necesaria una nueva categoría estable.

Antes de crear una nueva área debe comprobarse que:

1. ninguna de las áreas existentes representa adecuadamente la decisión;
2. el nuevo ámbito tiene significado propio desde el punto de vista de ingeniería;
3. no representa simplemente una herramienta, tecnología o producto;
4. previsiblemente podrá contener múltiples decisiones;
5. su significado puede mantenerse estable en el tiempo.

La taxonomía debe evolucionar a partir de necesidades reales.

### 3.17 Principio de evolución

IASI no pretende construir anticipadamente una clasificación completa de todas las decisiones de ingeniería posibles.

Las áreas se incorporarán cuando aparezcan decisiones reales que justifiquen su existencia.

**Las áreas no se inventan por anticipado. Se descubren cuando las decisiones de ingeniería las hacen necesarias.**

### 3.18 Regla fundamental

**nn clasifica la naturaleza principal de la decisión. Los metadatos describen todo lo demás.**

El identificador debe permitir saber dónde buscar una decisión.

El contenido y los metadatos explican su alcance, sus relaciones y sus consecuencias.

# **Part I**

## **General**

Esta sección reúne los Engineering Decision Records clasificados en el área **General** (00).

ID	Título
<a href="#">EDR-00001</a>	Registro único de decisiones de ingeniería
<a href="#">EDR-00002</a>	Identificación y clasificación de los EDR
<a href="#">EDR-00003</a>	Partir de la necesidad y no de las herramientas disponibles
<a href="#">EDR-00004</a>	Encapsular la complejidad interna
<a href="#">EDR-00005</a>	Automatizar lo repetible y mantener manual lo excepcional
<a href="#">EDR-00006</a>	Limitar la velocidad del sistema a la capacidad humana sostenible
<a href="#">EDR-00007</a>	Organizar la obra de IASI en tres volúmenes
<a href="#">EDR-00008</a>	Separar el proceso vivido del resultado consolidado
<a href="#">EDR-00009</a>	Hacer emerger las prácticas desde necesidades reales
<a href="#">EDR-00010</a>	Adoptar desarrollo dirigido por especificaciones
<a href="#">EDR-00011</a>	Lab execution model
<a href="#">EDR-00012</a>	Organizar el flujo del repositorio mediante nombres semánticos
<a href="#">EDR-00013</a>	Persistir el conocimiento de ingeniería en artefactos y no en el chat
<a href="#">EDR-00014</a>	Asociar el Sistema Inteligente al ingeniero y no al equipo
<a href="#">EDR-00015</a>	Separar roles, personas y puestos
<a href="#">EDR-00016</a>	Adoptar TOML como formato preferente de configuración

## 4 Registro único de decisiones de ingeniería

### 4.1 Contexto

Las decisiones de IASI aparecen durante el desarrollo de libros, herramientas, infraestructura, experimentos y conversaciones de diseño. Si cada producto conserva su propia versión de una decisión, el razonamiento termina duplicado y puede divergir.

### 4.2 Decisión

`iasi-edr` será el registro único de las decisiones de ingeniería de IASI. Los libros, artefactos, productos y el Journey pueden explicar o referenciar una decisión, pero no mantienen copias independientes de su contenido normativo.

### 4.3 Alternativas consideradas

- Mantener ADR o notas de decisión dentro de cada repositorio.
- Documentar únicamente el estado final en los libros.
- Conservar las decisiones solo en el Journey.

### 4.4 Consecuencias

Existe un punto único para consultar por qué se tomó una decisión. Se reduce la duplicación y se hace posible referenciar una identidad estable desde cualquier pieza del ecosistema.

## 5 Identificación y clasificación de los EDR

### 5.1 Contexto

Un registro central necesita una identidad estable y una clasificación suficientemente amplia para localizar decisiones funcionales, arquitectónicas, operativas, de datos, calidad e integración sin reducir EDR a decisiones puramente arquitectónicas.

### 5.2 Decisión

Los documentos usarán el identificador EDR-**nn**mmm, donde **nn** representa el área principal y **mmm** la secuencia dentro de esa área. 00 queda reservado para decisiones generales o transversales. Las áreas son: 00 General, 10 Funcional, 20 Arquitectura, 30 Constraints, 40 Datos, 50 Integración, 60 Seguridad, 70 Calidad, 80 Operación y 99 Pendiente de clasificar. Los identificadores definitivos son permanentes y no se reutilizan.

### 5.3 Alternativas consideradas

- Una secuencia global sin clasificación.
- Identificadores derivados únicamente del nombre del archivo.
- Numeración por repositorio o producto.

### 5.4 Consecuencias

El propio identificador aporta una primera señal de clasificación, permite ordenar documentos y mantiene referencias estables a lo largo del ecosistema.

## **6 Partir de la necesidad y no de las herramientas disponibles**

### **6.1 Contexto**

Empezar preguntando cómo resolver un problema con las herramientas actuales condiciona el espacio de solución antes de haber entendido la necesidad. La capacidad disponible termina dictando el diseño.

### **6.2 Decisión**

Ante un problema se preguntará primero qué se necesita para resolverlo. Solo después se evaluará si herramientas, tecnologías o prácticas existentes satisfacen esa necesidad. Si no lo hacen, se adaptarán o se crearán los mecanismos necesarios.

### **6.3 Alternativas consideradas**

- Elegir primero una herramienta y adaptar el problema a sus capacidades.
- Limitar la solución al catálogo tecnológico ya disponible.

### **6.4 Consecuencias**

Las decisiones se justifican por la necesidad que resuelven y no por familiaridad tecnológica. La metodología mantiene abierto el espacio de diseño y obliga a cuestionar incluso las soluciones aparentemente establecidas.

## **7 Encapsular la complejidad interna**

### **7.1 Contexto**

Una solución puede requerir mecanismos internos complejos sin que esa complejidad deba trasladarse a quien la utiliza. Exponer cada detalle técnico convierte la potencia interna en fricción operativa.

### **7.2 Decisión**

Los productos IASI deben encapsular la complejidad técnica detrás de interfaces, comandos y contratos simples. La complejidad se admite donde sea necesaria internamente, pero no se considera aceptable como coste impuesto al usuario.

### **7.3 Alternativas consideradas**

- Exponer directamente todos los componentes internos.
- Simplificar eliminando capacidades necesarias.

### **7.4 Consecuencias**

El diseño de interfaces pasa a ser parte de la ingeniería, no una capa cosmética. Una implementación compleja puede seguir ofreciendo un uso pequeño, explícito y predecible.

## **8 Automatizar lo repetible y mantener manual lo excepcional**

### **8.1 Contexto**

No toda tarea merece convertirse en un sistema general. Algunas aparecen con frecuencia y tienen una estructura clara; otras son excepcionales, visuales o requieren iteración específica.

### **8.2 Decisión**

IASI automatizará los casos repetibles, estructurables y suficientemente frecuentes. Los casos excepcionales permanecerán manuales o se resolverán con herramientas especializadas cuando aparezcan.

### **8.3 Alternativas consideradas**

- Automatizar desde el principio cualquier caso imaginable.
- Renunciar a automatizar y resolver todo manualmente.

### **8.4 Consecuencias**

Se reduce la superficie de mantenimiento y la complejidad accidental. La automatización crece cuando existe evidencia de repetición, no como anticipación especulativa.

## **9 Limitar la velocidad del sistema a la capacidad humana sostenible**

### **9.1 Contexto**

La disponibilidad continua de agentes elimina esperas que antes actuaban como pausas naturales. El humano puede encadenar decisiones de alta intensidad mientras la IA resuelve tareas paralelas, incluso cuando su capacidad de comprender y validar ya está disminuyendo.

### **9.2 Decisión**

La velocidad sostenible de IASI se diseñará alrededor de la capacidad humana para comprender, decidir, validar y asimilar, no alrededor de la velocidad máxima de los agentes. No se confundirá disponibilidad de la IA con disponibilidad humana.

### **9.3 Alternativas consideradas**

- Maximizar siempre el paralelismo de agentes.
- Medir productividad solo por volumen o tiempo de ejecución.

### **9.4 Consecuencias**

Los workflows deben admitir pausas, límites y momentos de asimilación. El rendimiento se evalúa por la calidad sostenible de las decisiones, no por mantener ocupados todos los agentes.

# 10 Organizar la obra de IASI en tres volúmenes

## 10.1 Contexto

El material de IASI mezcla fundamentos, historia real de construcción y metodología consolidada. Un único volumen obliga a mezclar el porqué, el proceso y el resultado final.

## 10.2 Decisión

La obra se separará en tres volúmenes: Volumen I para fundamentos y teoría, Volumen II para el viaje, errores y evolución, y Volumen III para la metodología o framework consolidado. Las partes se presentan con numeración romana y los capítulos mantienen numeración arábica.

## 10.3 Alternativas consideradas

- Un único libro cronológico.
- Un libro puramente metodológico que oculte el proceso real.
- Repartir el contenido por tecnologías en lugar de por función narrativa.

## 10.4 Consecuencias

Cada volumen puede mantener una voz y propósito propios sin perder coherencia. El lector puede distinguir teoría, evidencia del proceso y método final.

# 11 Separar el proceso vivido del resultado consolidado

## 11.1 Contexto

Las metodologías suelen describir únicamente el resultado final, pero IASI también quiere conservar el proceso efectivo: discusiones, errores, cambios de criterio y descubrimientos que llevaron a ese estado.

## 11.2 Decisión

El Book contiene la versión final y correcta del conocimiento. *iasi-journey* conserva cómo se llegó a ella. Las entradas del Journey se tratan como registros inmutables del proceso y no se reescriben para hacer que el pasado parezca coherente con la decisión final.

## 11.3 Alternativas consideradas

- Reescribir el Journey cuando cambia la conclusión.
- Incluir toda la conversación histórica dentro del Book.

## 11.4 Consecuencias

IASI puede mostrar tanto el conocimiento consolidado como el proceso real que lo produjo. La discrepancia histórica deja de ser ruido y se convierte en evidencia de evolución.

# **12 Hacer emerger las prácticas desde necesidades reales**

## **12.1 Contexto**

IASI es a la vez metodología y caso de estudio de su propia construcción. Introducir de antemano skills, especificaciones, agentes o procesos completos convertiría el proyecto en una demostración prefabricada.

## **12.2 Decisión**

Las prácticas, skills, automatizaciones y mecanismos de IASI deben surgir cuando una necesidad real del propio ecosistema los justifique. IASI se utilizará como caso de estudio vivo para validar esas decisiones.

## **12.3 Alternativas consideradas**

- Diseñar por adelantado el framework completo y después buscar ejemplos.
- Adoptar una práctica porque sea habitual en otros ecosistemas.

## **12.4 Consecuencias**

Cada componente puede señalar la necesidad concreta que lo originó. El framework final se apoya en experiencia acumulada y no solo en diseño teórico.

# 13 Adoptar desarrollo dirigido por especificaciones

## 13.1 Contexto

La reducción del coste de implementación aumenta el valor relativo de decidir y especificar correctamente qué debe construirse. La implementación, humana o asistida por IA, no debe convertirse en el lugar donde se descubre implícitamente el comportamiento esperado.

## 13.2 Decisión

IASI adopta desarrollo dirigido por especificaciones. La especificación explícita es un artefacto principal y precede a la implementación cuando el cambio lo requiere. OpenSpec se utiliza como mecanismo para materializar este enfoque allí donde resulte aplicable.

## 13.3 Alternativas consideradas

- Implementar primero y documentar después.
- Confiar en prompts o conversaciones efímeras como única especificación.

## 13.4 Consecuencias

Las implementaciones pueden cambiar de herramienta o agente sin perder la intención. La revisión se centra primero en qué debe ocurrir y después en cómo se implementa.

## **14 Lab execution model**

# 15 Lab execution model

## 15.1 Contexto

Algunos labs de IASI pueden resolverse como ejercicios pequeños y autocontenidos. Otros, sin embargo, deben trabajar sobre **proyectos reales**, con repositorios propios, historia Git, código, configuración y evolución independiente.

`iasi-graphics` es un ejemplo de este segundo caso.

El objetivo del lab no es simplemente construir el proyecto final. El objetivo es **observar y validar cómo IASI avanza a medida que recibe información**, qué puede hacer en cada momento, qué debe rechazar por falta de información y cómo transforma sucesivos inputs en decisiones, definitions, planes y resultados.

El lab debe permitir repetir ese proceso de forma controlada y reproducible.

## 15.2 Decisión

Los labs que trabajen sobre proyectos reales se ejecutarán sobre el **repositorio real del proyecto**.

El lab define el experimento y su secuencia de pasos. El repositorio contiene el estado real producido durante su ejecución.

Book II / Lab

define y explica

secuencia de pasos

inputs sucesivos

repositorio real

estado paso 1

estado paso 2

```
estado paso 3
...
```

No se mantendrá una copia paralela del proyecto dentro del lab.

## 15.3 El lab como secuencia

Un lab no representa únicamente un resultado final.

Representa una **secuencia controlada de estados**.

Por ejemplo, un lab sobre **iasi-graphics** podría comenzar con un input externo:

```
Queremos hacer un filtro y un MCP que genere gráficos.
```

Con esa información, IASI puede entender parte de la intención, pero no dispone necesariamente de información suficiente para diseñar o implementar la solución.

El comportamiento correcto puede ser detenerse y declarar que existen huecos que impiden continuar.

Posteriormente puede recibirse un nuevo input interno:

```
Por ahora únicamente crea el proyecto.
```

Esta nueva instrucción sí permite realizar una acción concreta: crear el proyecto, aunque siga sin existir información suficiente para diseñar el sistema completo.

El lab continúa de esta forma:

```
input
↓
análisis
↓
estado de conocimiento
↓
acción posible o bloqueo
↓
resultado
↓
nuevo input
↓
...
```

Cada paso valida que IASI actúa únicamente con la información disponible en ese momento.

IASI no debe anticipar decisiones futuras ni completar mediante suposiciones aquello que todavía no ha sido definido.

## 15.4 Separación entre lab y proyecto

El lab y el proyecto tienen responsabilidades diferentes.

### 15.4.1 El lab contiene

- la pregunta o propósito del experimento;
- la secuencia de pasos;
- los inputs que se introducen en cada paso;
- las condiciones esperadas;
- aquello que IASI puede hacer;
- aquello que IASI todavía no debe hacer;
- las evidencias y conclusiones del experimento.

### 15.4.2 El repositorio contiene

- el proyecto real;
- los inputs incorporados durante la ejecución;
- las definiciones producidas;
- los planes;
- el código;
- la configuración;
- los artefactos generados;
- la historia Git resultante.

Por tanto:

**El lab controla el experimento; el repositorio contiene el proyecto.**

## 15.5 Estados oficiales del lab

Cada paso significativo tendrá asociado un **estado oficial reproducible** del repositorio.

Estos estados se identificarán mediante tags Git.

Por ejemplo:

```
lab-graphics-step-01
lab-graphics-step-02
lab-graphics-step-03
```

Cada tag identifica el estado exacto del proyecto correspondiente a un determinado punto del experimento.

Los tags forman parte del repositorio oficial utilizado para ejecutar el lab.

No representan la historia particular de cada usuario.

Representan los **checkpoints oficiales del experimento**.

## 15.6 Identidad y orden editorial

El número utilizado para ordenar un lab dentro de Book II no forma parte de su identidad.

Por ejemplo:

```
005-graphics/
```

puede pasar posteriormente a:

```
010-graphics/
```

sin que la identidad del lab cambie.

Por esta razón, los tags no incorporarán el número editorial.

Se utilizará:

```
lab-graphics-step-01
```

y no:

```
lab-005-step-01
```

Esto evita que una reorganización editorial obligue a modificar la historia Git de los proyectos.

El principio es:

**El número ordena. El nombre identifica.**

## 15.7 Historia del usuario

Una vez iniciado el lab, el repositorio pertenece operativamente al usuario.

El usuario puede:

- modificar archivos;
- experimentar;
- cometer errores;
- crear sus propios commits;
- crear ramas;
- realizar soluciones alternativas;
- desviarse de la secuencia prevista.

IASI Labs no necesita controlar esa historia.

Los commits creados durante el trabajo son parte de la experiencia del usuario.

Los tags oficiales permanecen como referencias estables.

Por tanto:

**Los commits son del usuario; los tags son del lab.**

## 15.8 Regreso a un checkpoint

Si el usuario se pierde, rompe el proyecto o desea repetir una parte del experimento, puede regresar a cualquiera de los estados oficiales.

Por ejemplo:

```
git switch --detach lab-graphics-step-05
```

El repositorio volverá al estado exacto correspondiente a ese paso.

Desde ahí puede:

- inspeccionar el estado;
- repetir el paso;
- crear una nueva rama;
- continuar el experimento de otra forma.

El usuario puede haber realizado posteriormente otros labs o haber avanzado mucho más en el repositorio.

Eso no impide volver a un checkpoint anterior.

Por ejemplo, puede haber completado:

```
lab-graphics-step-05
lab-mcp-step-03
```

y regresar conscientemente a:

```
lab-graphics-step-05
```

El estado posterior dejará de estar visible en ese checkout, pero no desaparece.

Puede recuperarse nuevamente mediante su tag correspondiente.

Los tags funcionan así como **puntos de restauración oficiales del recorrido experimental**.

## 15.9 Reproducibilidad

Este modelo permite que distintas personas ejecuten el mismo lab manteniendo historias Git completamente diferentes.

Dos usuarios pueden partir del mismo checkpoint:

```
lab-graphics-step-03
```

y producir soluciones diferentes.

El lab sigue siendo reproducible porque ambos conocen exactamente el estado inicial desde el que parte el paso.

La reproducibilidad no exige que todos produzcan los mismos commits.

Exige que los puntos de partida oficiales sean conocidos y recuperables.

## 15.10 Consecuencias

Este modelo permite que un lab evolucione naturalmente hacia un proyecto real sin necesidad de mantener una versión artificial específica para formación.

El mismo repositorio puede ser simultáneamente:

- un proyecto real de IASI;
- el sujeto de uno o varios labs;
- una fuente de evidencias sobre el funcionamiento de la metodología;
- un entorno donde el usuario puede experimentar libremente.

El lab no simula el desarrollo del proyecto.

**El lab utiliza el desarrollo real como experimento.**

Esto permite probar no solo el resultado final de IASI, sino su comportamiento durante todo el proceso: cuándo puede avanzar, cuándo debe detenerse, qué información utiliza y cómo cambia el proyecto conforme aparecen nuevos inputs.

## 15.11 Principios derivados

**El lab controla el experimento; el repositorio contiene el proyecto.**

**Los commits son del usuario; los tags son del lab.**

**El número ordena; el nombre identifica.**

**Cada paso del lab debe poder partir de un estado oficial conocido.**

**IASI debe actuar únicamente con la información disponible en cada paso.**

**Los labs no simulan proyectos reales: pueden ejecutarse directamente sobre ellos.**

## **16 Organizar el flujo del repositorio mediante nombres semánticos**

# 17 Organizar el flujo del repositorio mediante nombres semánticos

## 17.1 Contexto

Durante la definición inicial del flujo de trabajo se utilizaron prefijos alfabéticos para ordenar visualmente las áreas principales del repositorio:

```
A-inputs/
B-engine/
C-outputs/
D-deliverables/
```

Esta estructura hacía visible una secuencia aproximada de trabajo, pero introducía dos problemas.

En primer lugar, los prefijos A-, B-, C- y D- no aportan significado propio. Su función es únicamente forzar el orden de las carpetas.

En segundo lugar, ese orden puede sugerir un proceso estrictamente secuencial por fases, cuando el trabajo real puede volver sobre los inputs, revisar especificaciones, regenerar outputs o repetir validaciones tantas veces como sea necesario.

Además, algunos nombres resultaban demasiado ambiguos. En particular, **engine/** podía interpretarse como un componente software ejecutable, mientras que su contenido representa realmente los elementos que coordinan y gobiernan el proceso de trabajo.

## 17.2 Decisión

Las áreas principales del repositorio se nombrarán por su **función**, sin prefijos artificiales destinados únicamente a imponer un orden visual.

La estructura canónica será:

```
inputs/
orchestration/
outputs/
releases/
```

Cada área representa una responsabilidad diferente:

- **inputs/** contiene la información que entra en el proceso.
- **orchestration/** contiene los elementos que organizan, dirigen y gobiernan el trabajo realizado sobre los inputs, como especificaciones, tareas y otros mecanismos de coordinación.
- **outputs/** contiene los artefactos producidos durante el proceso.
- **releases/** contiene los artefactos aceptados y preparados para su publicación, entrega o distribución.

La estructura no representa fases rígidas ni obliga a recorrer las carpetas una sola vez en ese orden.

## 17.3 Rationale

Los nombres deben expresar **qué es cada cosa**, no únicamente dónde queremos que aparezca en un árbol de directorios.

Los prefijos de orden convierten una decisión de presentación en parte permanente del modelo del repositorio. Al eliminarlos, la estructura queda definida por conceptos que pueden evolucionar sin depender de una numeración o secuencia artificial.

**orchestration/** sustituye a **engine/** porque describe mejor la responsabilidad de su contenido. No representa necesariamente un motor ejecutable, sino el conjunto de artefactos que coordinan el proceso, actualmente incluyendo elementos como:

```
orchestration/
 specifications/
 tasks/
```

y pudiendo incorporar en el futuro otros elementos de gobierno o coordinación sin alterar el significado del área.

**releases/** sustituye a **deliverables/** para distinguir entre aquello que el proceso **produce** y aquello que finalmente se **acepta y libera**.

## 17.4 Alternativas consideradas

### 17.4.1 Mantener los prefijos A-, B-, C-, D-

Se descarta porque fuerza un orden visual artificial y puede interpretarse como una metodología secuencial por fases.

### 17.4.2 Mantener `engine/`

Se descarta porque puede confundirse con un componente ejecutable y no describe con suficiente precisión el papel de especificaciones, tareas y futuros mecanismos de coordinación.

### 17.4.3 Utilizar `model/`

Se consideró como representación interna del problema, pero se descarta por su ambigüedad en un contexto de Sistemas Inteligentes, donde `model` se asocia de forma natural a modelos de IA.

### 17.4.4 Utilizar `deliverables/`

Se descarta a favor de `releases/`, que expresa mejor que los artefactos contenidos en esa área han superado el proceso de aceptación y están preparados para salir del flujo de trabajo.

## 17.5 Consecuencias

La estructura principal del repositorio pasa a ser legible sin depender de prefijos de orden.

El flujo puede entenderse conceptualmente como:

```
inputs
 ↓
orchestration
 ↓
outputs
 ↓
releases
```

sin que este esquema implique una ejecución estrictamente lineal.

Las herramientas, agentes y personas podrán volver sobre cualquiera de estas áreas cuando el proceso lo requiera.

La distinción entre outputs/ y releases/ hace explícito que **producir un artefacto no implica automáticamente aceptarlo.**

## 17.6 Principios derivados

Los nombres expresan función, no posición.

El orden visual no debe convertirse en una falsa secuencia metodológica.

Un output es producido; un release ha sido aceptado para salir del proceso.

## **18 Persistir el conocimiento de ingeniería en artefactos y no en el chat**

# 19 Persistir el conocimiento de ingeniería en artefactos y no en el chat

## 19.1 Contexto

Durante el trabajo asistido por Sistemas Inteligentes es natural utilizar la conversación como espacio de análisis, exploración y toma de decisiones. Sin embargo, si el conocimiento relevante permanece únicamente en el chat, queda ligado a una sesión concreta y resulta difícil versionarlo, revisarlo, reutilizarlo, procesarlo o convertirlo en fuente de verdad para otros agentes y herramientas.

Escribir una decisión en un documento puede requerir prácticamente el mismo esfuerzo que expresarla en una conversación, pero su valor operativo es muy distinto.

## 19.2 Decisión

El chat se utilizará principalmente para **pensar, discutir y llegar a decisiones**.

El conocimiento de ingeniería que deba persistir se registrará en documentos o artefactos explícitos, aunque su contenido sea mínimo.

Los artefactos persistentes serán la fuente reutilizable para:

- versionado;
- trazabilidad;
- revisión;
- validación;
- reutilización;
- procesamiento automático;
- rematerialización de outputs;
- trabajo posterior de agentes y herramientas.

El prompt o la conversación no se utilizarán como contenedor principal del conocimiento del proyecto.

## 19.3 Rationale

La conversación es un excelente medio de elaboración, pero un mal mecanismo de persistencia estructurada.

Un artefacto explícito permite separar el proceso mental que llevó a una decisión de la decisión que finalmente debe gobernar el sistema.

Esto evita que un agente necesite reconstruir contexto histórico para conocer algo que ya fue decidido.

## 19.4 Alternativas consideradas

### 19.4.1 Mantener el conocimiento principal en prompts o conversaciones

Se descarta porque dificulta versionado, trazabilidad, revisión y reutilización, y obliga a recuperar contexto conversacional para reproducir decisiones anteriores.

### 19.4.2 Documentar únicamente decisiones grandes

Se descarta como regla general. Una decisión pequeña puede ser igualmente relevante para una ejecución posterior y su coste de persistencia puede ser mínimo.

## 19.5 Consecuencias

Los proyectos IASI tenderán a disponer de más artefactos pequeños y explícitos.

Los agentes deberán trabajar sobre esos artefactos como entradas de ingeniería, en lugar de depender de memoria conversacional implícita.

La conversación seguirá siendo parte del proceso vivido y podrá conservarse en Journey u otros registros, pero no sustituirá al conocimiento persistido que gobierna el proyecto.

**La conversación sirve para pensar; los artefactos conservan lo decidido.**

## **20 Asociar el Sistema Inteligente al ingeniero y no al equipo**

# 21 Asociar el Sistema Inteligente al ingeniero y no al equipo

## 21.1 Contexto

En un equipo aumentado podría imaginarse un único Sistema Inteligente compartido que aumentase colectivamente al equipo. Sin embargo, el uso real muestra que el SI trabaja ligado a la persona: conoce su contexto, recibe sus instrucciones, participa en sus decisiones y termina reflejando su forma de trabajar.

Incluso dos personas que utilicen el mismo modelo no disponen funcionalmente del mismo SI, porque sus conversaciones, contexto, preferencias y hábitos divergen.

## 21.2 Decisión

En IASI, el Sistema Inteligente se considera asociado al **ingeniero individual**.

El equipo está formado por **ingenieros aumentados**, no por un conjunto de personas alrededor de una única IA compartida.

La coordinación, asignación y delegación de trabajo continúa produciéndose entre personas. Cada ingeniero decide cómo utilizar su propio SI para realizar la tarea que le corresponde.

El SI aumenta a la persona, pero no sustituye la relación organizativa entre personas ni asume por sí mismo la responsabilidad profesional del trabajo.

## 21.3 Rationale

La relación humano-SI acumula contexto y formas de trabajo específicas. Tratar el SI como recurso indistinto del equipo ocultaría esa dependencia y dificultaría razonar sobre responsabilidad, sesgos, autoridad y calidad.

El modelo permite mantener una unidad clara de responsabilidad: el ingeniero aumentado puede utilizar capacidades inteligentes sin diluir su papel dentro de una entidad colectiva ambigua.

## **21.4 Alternativas consideradas**

### **21.4.1 Un SI compartido por todo el equipo**

Puede existir técnicamente, pero no se adopta como modelo organizativo base porque mezcla contextos individuales y dificulta atribuir decisiones y responsabilidades.

### **21.4.2 Tratar el SI como un miembro autónomo del equipo**

Se descarta como modelo general. El SI participa activamente en la ingeniería, pero sus capacidades, permisos y autoridad deben quedar diseñados y gobernados.

## **21.5 Consecuencias**

Dos ingenieros aumentados pueden producir comportamientos distintos incluso utilizando el mismo modelo.

Los mecanismos de trazabilidad deberán poder distinguir qué persona, qué SI y qué contexto participaron en una decisión o materialización.

También debe asumirse que la amplificación no afecta solo a capacidades positivas: hábitos, sesgos o defectos de trabajo del humano pueden igualmente reforzarse.

**La Ingeniería Aumentada amplifica capacidades, pero también puede amplificar defectos.**

## **22 Separar roles, personas y puestos**

## 23 Separar roles, personas y puestos

### 23.1 Contexto

Las estructuras organizativas suelen confundir tres conceptos diferentes: la función que debe existir, la persona que la desempeña y el puesto o recurso organizativo asignado para cubrirla.

Esta confusión lleva a diseñar estructuras innecesariamente rígidas y a suponer que cada función requiere siempre una persona o puesto exclusivo.

### 23.2 Decisión

IASI distinguirá explícitamente **rol**, **persona** y **puesto**.

Los roles representan responsabilidades y puntos de vista que deben existir para que la ingeniería pueda realizarse correctamente.

Una misma persona puede desempeñar varios roles y un mismo rol puede distribuirse entre varias personas según el contexto.

Al tomar una decisión debe quedar claro desde qué rol se está actuando. El hecho de que una persona pueda ejercer varios roles no implica que deba mezclarlos de forma inconsciente en una misma decisión.

La estructura organizativa concreta se adapta a los recursos disponibles sin eliminar las responsabilidades necesarias.

### 23.3 Rationale

Definir primero los roles permite razonar sobre responsabilidades antes de convertirlas prematuramente en organigramas, puestos o necesidades de personal.

Esto hace que el modelo sea aplicable tanto a equipos grandes como a proyectos pequeños donde una sola persona deba desempeñar varias funciones.

## **23.4 Alternativas consideradas**

### **23.4.1 Asociar un rol a un puesto fijo**

Se descarta porque convierte una responsabilidad metodológica en una decisión organizativa contingente.

### **23.4.2 Permitir que una persona actúe simultáneamente desde varios roles sin distinguirlos**

Se descarta porque dificulta saber qué responsabilidad, criterio o autoridad está guiando una decisión.

## **23.5 Consecuencias**

Las descripciones metodológicas de IASI deberán identificar responsabilidades por rol y no exigir de forma innecesaria estructuras organizativas concretas.

La asignación de personas y recursos podrá cambiar sin alterar la necesidad de las funciones fundamentales.

**Los roles son necesarios; los puestos y los recursos son contingentes.**

## **24 Adoptar TOML como formato preferente de configuración**

## 25 Adoptar TOML como formato preferente de configuración

### 25.1 Contexto

IASI necesita un criterio común para los ficheros de configuración creados y controlados por sus propios artefactos. Entre los formatos candidatos, **TOML** y **YAML** (`.yaml` o `.yml`) ofrecen buena legibilidad humana y amplio soporte de herramientas.

Sin embargo, responden a objetivos diferentes. YAML es un lenguaje general de serialización capaz de representar estructuras complejas y ofrecer mecanismos avanzados. TOML está orientado específicamente a configuración y favorece una sintaxis más explícita, limitada y predecible.

Para IASI, un fichero de configuración debe ser fácil de leer y modificar por una persona, sencillo de procesar automáticamente y, sobre todo, difícil de interpretar de forma inesperada.

### 25.2 Decisión

IASI adopta **TOML como formato preferente para la configuración propia del ecosistema**.

Cuando IASI controle el formato de un fichero de configuración nuevo, se utilizará `.toml` por defecto.

YAML no queda prohibido. Se utilizará cuando sea impuesto por una herramienta, protocolo o ecosistema externo, o cuando exista una razón técnica concreta que haga más adecuada su capacidad para representar estructuras de datos complejas.

La decisión se aplica transversalmente a IASI y no está asociada a un artefacto, repositorio o subsistema concreto.

## 25.3 Rationale

La preferencia por TOML no se basa en que sea más expresivo que YAML, sino precisamente en que es **deliberadamente menos expresivo** para el dominio de configuración.

La menor cantidad de mecanismos sintácticos reduce el espacio de interpretaciones posibles y favorece configuraciones explícitas, estables y fáciles de validar.

Criterio	TOML	YAML
<b>Orientación principal</b>	Configuración	Serialización general de datos
<b>Sintaxis</b>	Explícita y relativamente pequeña	Flexible y muy expresiva
<b>Indentación significativa</b>	No	Sí
<b>Tipos</b>	Explícitos y previsibles	Más dependientes de reglas de interpretación
<b>Estructuras jerárquicas</b>	Adecuadas, con mayor explicitud	Muy naturales y compactas
<b>Mecanismos avanzados</b>	Deliberadamente limitados	Aliases, tags y otras construcciones
<b>Riesgo de complejidad accidental</b>	Menor	Mayor
<b>Adecuación a configuración IASI</b>	<b>Preferente</b>	Condicional

La restricción se considera una propiedad positiva: una configuración no necesita convertirse en un pequeño lenguaje para cumplir su función.

**No elegimos TOML porque pueda expresar más que YAML, sino porque puede expresar menos y esas restricciones encajan mejor con lo que IASI necesita de una configuración.**

## 25.4 Alternativas consideradas

Alternativa	Ventajas	Inconvenientes	Decisión
<b>TOML</b>	Explícito, predecible, orientado a configuración y con tipado claro	Más verboso en estructuras profundamente jerárquicas	<b>Adoptada</b>

Alternativa	Ventajas	Inconvenientes	Decisión
<b>YAML</b>	Muy legible en estructuras jerárquicas, compacto y ampliamente utilizado	Mayor complejidad sintáctica y semántica; más formas de expresar lo mismo	No como formato propio por defecto
<b>JSON</b>	Universal, especificación pequeña y excelente interoperabilidad	Poco cómodo para edición humana y sin comentarios estándar	No adoptada para configuración humana

### 25.4.1 YAML como formato general

YAML sigue siendo apropiado cuando el problema consiste principalmente en describir estructuras de datos jerárquicas o cuando una tecnología externa lo establece como formato nativo.

Las extensiones `.yaml` y `.yml` representan el mismo formato y no constituyen alternativas diferentes dentro de esta decisión.

### 25.4.2 JSON como configuración

JSON es adecuado como formato de intercambio entre sistemas, pero no se adopta como formato preferente de configuración humana de IASI. La ausencia de comentarios estándar y su sintaxis más rígida lo hacen menos conveniente para ficheros mantenidos directamente por personas.

## 25.5 Consecuencias

Los nuevos formatos de configuración definidos por IASI deberán considerar TOML como primera opción.

Los artefactos existentes no deberán migrarse mecánicamente solo para uniformar extensiones. La migración tendrá sentido cuando IASI controle realmente el formato y el cambio aporte valor suficiente.

Cuando una herramienta externa requiera YAML, JSON u otro formato, IASI utilizará el formato requerido sin introducir conversiones artificiales únicamente para cumplir esta preferencia.

Las excepciones deberán responder a una necesidad técnica identificable y no simplemente a la costumbre de utilizar YAML para cualquier configuración.

La elección del formato de serialización tampoco convierte a TOML en el modelo de dominio. Cuando exista un modelo interno, TOML será una representación de entrada o persistencia de dicho modelo, no necesariamente su definición conceptual.

**Configuración propia de IASI → TOML por defecto. Formato impuesto o necesidad específica → el formato que corresponda.**

# **Part II**

# **Funcional**

Esta sección reúne los Engineering Decision Records clasificados en el área **Funcional** (10).

ID	Título
<a href="#">EDR-10001</a>	Usar iasi-home como puerta de entrada visual al ecosistema
<a href="#">EDR-10002</a>	Distinguir libros de productos y artefactos en iasi-home
<a href="#">EDR-10003</a>	Ofrecer iasi-home en castellano e inglés con detección automática
<a href="#">EDR-10004</a>	Generar un catálogo por área de EDR
<a href="#">EDR-10005</a>	Crear EDR mediante un Addin de RStudio
<a href="#">EDR-10006</a>	Crear una presentación de introducción a IASI
<a href="#">EDR-10007</a>	Proporcionar un agente IASI capaz de ejecutar trabajo sobre el workspace
<a href="#">EDR-10008</a>	Distinguir productos y subproductos en iasi-home
<a href="#">EDR-10009</a>	Congelar iasi-home por cobertura y navegación, no por completitud de contenido

## 26 Usar iasi-home como puerta de entrada visual al ecosistema

### 26.1 Contexto

IASI ya no es un único libro con repositorios auxiliares, sino un ecosistema con varios libros, productos y artefactos. Hace falta un punto de entrada que permita orientarse sin conocer previamente su estructura.

### 26.2 Decisión

`iasi-home` será la portada navegable del ecosistema IASI y concentrará el acceso visual a sus principales piezas.

### 26.3 Alternativas consideradas

- Usar GitHub como única página de entrada.
- Hacer que cada libro funcione como portada del ecosistema.

### 26.4 Consecuencias

La navegación comienza por el mapa del ecosistema y después desciende a cada pieza. `iasi-home` no necesita absorber el contenido de los productos, solo presentarlos y enlazarlos.

## **27 Distinguir libros de productos y artefactos en iasi-home**

### **27.1 Contexto**

Los libros y los repositorios reutilizables cumplen funciones diferentes. Presentarlos como una lista homogénea oculta esa diferencia y dificulta entender qué se lee y qué se usa.

### **27.2 Decisión**

*iasi-home* distinguirá explícitamente los libros de los productos, artefactos o repositorios reutilizables. La estructura visual debe mostrar ambas familias sin confundir publicación editorial con herramienta.

### **27.3 Alternativas consideradas**

- Una única lista de repositorios.
- Clasificar todo como documentación.

### **27.4 Consecuencias**

El visitante entiende antes la naturaleza de cada pieza y puede elegir si quiere leer, explorar una metodología o utilizar un producto.

## 28 Ofrecer iasi-home en castellano e inglés con detección automática

### 28.1 Contexto

La landing necesita versiones en castellano e inglés sin duplicar toda la arquitectura del sitio ni obligar al visitante a escoger idioma antes de entrar.

### 28.2 Decisión

La raíz mantendrá `index_es.qmd` e `index_en.qmd`. `index.qmd` actuará como entrada de detección y el JavaScript común resolverá el idioma al cargar, permitiendo además el cambio explícito entre versiones. Los recursos compartidos permanecerán fuera de árboles específicos de idioma.

### 28.3 Alternativas consideradas

- Mantener una jerarquía completa `pages/en/`.
- Publicar solo un idioma.
- Duplicar recursos y páginas comunes por idioma.

### 28.4 Consecuencias

La estructura bilingüe permanece pequeña y predecible. Las páginas comunes pueden tener variantes de idioma sin replicar todo el sitio.

## 29 Generar un catálogo por área de EDR

### 29.1 Contexto

Cada área de EDR necesita una entrada navegable incluso cuando todavía no contiene decisiones. Mantener manualmente listas de documentos introduce desajustes en cuanto se crea, mueve o renombra un registro.

### 29.2 Decisión

Cada directorio de área tendrá un `index.qmd` generado automáticamente. El documento incluirá siempre una introducción que identifique el área y, cuando existan registros, una tabla simple ID | Título con enlaces. Si el área está vacía, mostrará un mensaje explícito en lugar de quedar sin contenido.

### 29.3 Alternativas consideradas

- Mantener el catálogo a mano.
- No crear páginas de área hasta que exista el primer EDR.
- Mostrar metadatos completos en el catálogo.

### 29.4 Consecuencias

Todas las áreas son navegables desde el primer día y el catálogo refleja siempre los documentos realmente existentes.

## 30 Crear EDR mediante un Addin de RStudio

### 30.1 Contexto

Crear un EDR a mano obliga a recordar área, secuencia, nombre de archivo, ruta y front matter. Es trabajo mecánico y además puede romper la consistencia del registro.

### 30.2 Decisión

La creación ordinaria de EDR se realizará mediante un Addin de RStudio apoyado en una plantilla. El usuario selecciona el área y proporciona el título; el sistema calcula el siguiente identificador, resuelve el directorio, genera el front matter y abre el documento creado.

### 30.3 Alternativas consideradas

- Crear archivos manualmente.
- Pedir al usuario que escriba el número de área y la secuencia.
- Mantener un contador externo separado de los documentos.

### 30.4 Consecuencias

El humano decide el contenido y la clasificación; el sistema mantiene la mecánica de identidad y ubicación. El flujo queda preparado para regenerar el índice de área después de crear el documento.

# 31 Crear una presentación de introducción a IASI

## 31.1 Contexto

IASI dispone de libros, documentación, repositorios, artefactos y una presencia web, pero esos medios no sustituyen una presentación concebida específicamente para **introducir IASI de forma progresiva y visual**.

La necesidad no es crear nueva documentación ni resumir un libro. Se necesita un artefacto que permita presentar IASI ante otras personas, explicar sus ideas fundamentales de forma secuencial y apoyar una exposición oral.

La presentación debe poder evolucionar a medida que evolucione la comprensión de IASI y debe formar parte del propio ecosistema de artefactos del proyecto.

## 31.2 Decisión

Se creará una presentación específica denominada **Introducing IASI**, desarrollada como el proyecto `introducing-iasi`.

Su función será introducir IASI progresivamente, priorizando la comunicación visual y la exposición de conceptos antes que la acumulación de texto explicativo.

La presentación será un artefacto propio de IASI y se desarrollará aplicando el propio enfoque IASI a su construcción.

## 31.3 Alternativas consideradas

- Utilizar directamente los libros de IASI como material de presentación.
- Utilizar `iasi-home` como sustituto de una presentación.
- Preparar un documento estático de introducción.
- No crear por ahora un artefacto específico de presentación.

Estas alternativas permiten explicar o documentar IASI, pero no proporcionan el tipo de narrativa secuencial, visual y orientada a exposición que requiere una presentación.

## 31.4 Consecuencias

IASI dispone de un artefacto específico para su presentación pública.

La narrativa de introducción puede evolucionar independientemente de los libros, la documentación y el portal web.

`introducing-iasi` se convierte además en un caso real sobre el que aplicar y validar IASI durante la construcción de un nuevo artefacto.

La tecnología utilizada para materializar la presentación se decide de forma independiente en EDR-20013.

## 32 Proporcionar un agente IASI capaz de ejecutar trabajo sobre el workspace

### 32.1 Contexto

La infraestructura local de IASI ya permite ejecutar modelos mediante Ollama.

Sin embargo, disponer de un modelo local no equivale a disponer de un agente capaz de realizar trabajo de ingeniería.

Un modelo utilizado en modo chat o prompt puede recibir contexto y generar una respuesta, pero no dispone necesariamente de las capacidades necesarias para trabajar sobre un proyecto real:

```
prompt
↓
model
↓
response
```

Para ejecutar trabajo IASI se necesita una capa adicional capaz de operar sobre el workspace.

La necesidad se hizo explícita al intentar utilizar los modelos locales como alternativa de ejecución: el modelo estaba disponible, pero faltaba el componente capaz de leer el proyecto, comprender una tarea, utilizar herramientas, modificar los artefactos permitidos y materializar resultados.

La existencia de agentes externos como Codex demostró además que la tarea y el ejecutor pueden separarse. Una misma tarea IASI puede ser ejecutada por distintos agentes sin redefinir el trabajo.

### 32.2 Decisión

IASI dispondrá de un agente denominado inicialmente `iasi-agent`, capaz de ejecutar trabajo directamente sobre un workspace IASI.

Su objetivo es proporcionar la capa de ejecución que convierte un modelo en un ejecutor de ingeniería.

Conceptualmente:

```
MODEL
↓
IASI AGENT
↓
WORKSPACE
```

`iasi-agent` deberá poder, según las reglas y permisos aplicables:

- comprender la tarea solicitada;
- descubrir e inspeccionar la estructura del proyecto;
- leer los artefactos relevantes;
- utilizar herramientas;
- ejecutar comandos cuando corresponda;
- crear o modificar únicamente los artefactos permitidos;
- materializar resultados;
- validar el trabajo realizado.

La capacidad de acceder al workspace no implica autorización para modificar cualquier contenido.

Las reglas de IASI siguen gobernando qué puede leerse, modificarse o producirse. Los artefactos definidos como inmutables continúan siendo inmutables independientemente del agente que ejecute el trabajo.

La ejecución local será una capacidad de primer nivel. `iasi-agent` deberá poder utilizar modelos locales cuando estos sean suficientes para la tarea.

## 32.3 Alternativas consideradas

### 32.3.1 Utilizar únicamente Ollama u Open WebUI

Permiten ejecutar e interactuar con modelos locales, pero no proporcionan por sí mismos el agente de workspace que necesita IASI.

Resuelven la inferencia y la conversación, no la ejecución completa del trabajo de ingeniería.

### 32.3.2 Depender exclusivamente de agentes externos

Agentes como Codex ya ofrecen excelentes capacidades sobre el workspace.

Sin embargo, hacer depender toda la ejecución de IASI de un único agente externo introduciría una dependencia innecesaria entre la metodología y un ejecutor concreto.

Además, disponibilidad, cuotas, conectividad o evolución del producto podrían impedir temporalmente la ejecución sin que haya cambiado la tarea.

### 32.3.3 Construir inmediatamente un agente completo desde cero

Proporcionaría control total, pero obligaría a desarrollar antes de tiempo capacidades ya resueltas por plataformas agentic existentes: gestión de modelos, herramientas, sesiones, permisos, edición de archivos, ejecución de comandos e integración con protocolos externos.

IASI necesita definir primero qué espera de un agente y reutilizar infraestructura existente siempre que satisfaga esa necesidad.

## 32.4 Consecuencias

El modelo deja de confundirse con el agente.

Ollama permanece como runtime de modelos y `iasi-agent` proporciona la capa de ejecución sobre el workspace.

IASI puede utilizar diferentes modelos sin redefinir el agente y diferentes agentes sin redefinir la tarea.

Las tareas permanecen independientes del ejecutor.

`iasi-agent` se convierte en uno de los posibles ejecutores de IASI, no en el workflow de IASI.

La arquitectura permite trabajar localmente cuando exista un modelo suficiente para la tarea y utilizar otros ejecutores cuando resulte más apropiado.

Se establece como principio:

**El agente es un ejecutor, no el workflow.**

### **33 Distinguir productos y subproductos en iasi-home**

## 34 Distinguir productos y subproductos en iasi-home

### 34.1 Contexto

IASI genera numerosas piezas durante su propia construcción. Algunas nacen únicamente como capacidades necesarias para el ecosistema. Otras, aunque también hayan sido creadas inicialmente para resolver necesidades internas, pueden adquirir autonomía y utilizarse fuera de IASI con los ajustes adecuados.

Presentar todas esas piezas bajo una única categoría oculta una diferencia útil para entender el ecosistema.

### 34.2 Decisión

iasi-home distinguirá entre **Productos** y **Subproductos**.

Se utilizará el siguiente criterio:

- **Subproductos:** cosas que IASI necesitó y creó durante su propia construcción.
- **Productos:** cosas creadas originalmente para IASI que, con los ajustes necesarios, pueden vivir y ser utilizadas fuera de IASI como elementos autónomos.

La diferencia no depende de quién motivó originalmente su creación, ya que ambas categorías pueden nacer de necesidades internas. La diferencia principal es su capacidad de adquirir vida propia fuera del ecosistema.

En la navegación principal, **Productos** aparecerá antes que **Subproductos**.

### 34.3 Rationale

La clasificación muestra primero las piezas con vocación de uso autónomo y después aquellas que explican cómo IASI se ha dotado de capacidades durante su desarrollo.

También evita utilizar **Artefactos** como categoría de navegación para productos concretos, reservando ese término para su significado de ingeniería.

## **34.4 Alternativas consideradas**

### **34.4.1 Mantener Artefactos como categoría principal**

Se descarta porque elementos como IASI Tools Dev, IASI RStudio, IASI Quarto o IASI Lua se perciben y se consumen mejor como productos que como artefactos metodológicos.

### **34.4.2 Considerar producto todo lo que IASI haya creado**

Se descarta porque eliminaría la diferencia entre capacidades internas surgidas durante la construcción y elementos que pueden emanciparse del ecosistema.

## **34.5 Consecuencias**

La navegación de `iasi-home` podrá presentar una imagen más clara de qué partes del ecosistema pueden utilizarse de forma autónoma y qué partes representan capacidades surgidas durante su construcción.

La clasificación podrá evolucionar a medida que un subproducto adquiera suficiente autonomía para convertirse en producto.

### **35 Congelar iasi-home por cobertura y navegación, no por completitud de contenido**

## 36 Congelar iasi-home por cobertura y navegación, no por completitud de contenido

### 36.1 Contexto

Congelar la web podía interpretarse como terminar previamente todos los libros, productos, manuales, subproductos y demás contenidos que aparecen en ella.

Ese criterio bloquearía indefinidamente la estabilización de `iasi-home`, ya que IASI continúa creciendo y muchas piezas estarán durante largos periodos en construcción.

La web ya dispone de páginas de estado como `status/to-be-done` y `status/under-construction`, por lo que puede representar explícitamente contenido todavía incompleto sin ocultarlo ni romper la navegación.

### 36.2 Decisión

`iasi-home` se considerará congelable cuando proporcione una **imagen global, suficientemente completa y navegable del ecosistema IASI**, aunque parte del contenido siga en construcción.

Para congelar la web se exigirá que:

- los menús y submenús representen las áreas relevantes del ecosistema;
- los enlaces funcionen;
- cada elemento importante tenga un destino razonable;
- los elementos todavía no desarrollados puedan dirigir a páginas de estado;
- la navegación permita comprender qué existe, qué se está construyendo y cómo encajan las piezas.

No se exigirá que todas las páginas tengan el mismo nivel de profundidad ni que todos los productos o documentos estén terminados.

## 36.3 Rationale

La web es una representación pública del sistema, no una prueba de que el sistema haya concluido su desarrollo.

Mostrar correctamente una pieza en construcción es preferible a ocultarla, siempre que el visitante pueda entender su función y su estado.

## 36.4 Alternativas consideradas

### 36.4.1 Esperar a terminar cada elemento antes de mostrarlo

Se descarta porque convertiría la congelación de la web en una dependencia de la finalización de todo IASI.

### 36.4.2 Mostrar únicamente las piezas terminadas

Se descarta porque ofrecería una imagen mutilada del alcance real del ecosistema.

## 36.5 Consecuencias

La prioridad inmediata de `iasi-home` pasa a ser la arquitectura de información, la navegación y la cobertura del ecosistema.

Los estados `to-be-done` y `under-construction` forman parte legítima de la representación pública mientras correspondan al estado real de la pieza.

**Congelar la web significa congelar una representación coherente del sistema, no terminar el sistema.**

**Part III**

**Arquitectura**

Esta sección reúne los Engineering Decision Records clasificados en el área **Arquitectura** (20).

ID	Título
<a href="#">EDR-20001</a>	Mantener iasi-edr como repositorio independiente
<a href="#">EDR-20002</a>	Publicar iasi-edr como Quarto book estructurado
<a href="#">EDR-20003</a>	Definir las áreas EDR en _iasi.yml
<a href="#">EDR-20004</a>	Organizar físicamente los EDR por área
<a href="#">EDR-20005</a>	Regenerar los índices EDR en pre-render
<a href="#">EDR-20006</a>	Mantener la lógica específica de EDR fuera de iasi.quarto
<a href="#">EDR-20007</a>	Separar la configuración común de los perfiles HTML y PDF
<a href="#">EDR-20008</a>	Usar la navbar como navegación principal de iasi-edr en HTML
<a href="#">EDR-20009</a>	Componer el estilo HTML con SCSS común y específico
<a href="#">EDR-20010</a>	Mantener iasi-graphics deliberadamente simple
<a href="#">EDR-20011</a>	Crear un espacio dedicado para capacidades agentic de IASI
<a href="#">EDR-20012</a>	Separar el entorno de laboratorios mediante una máquina virtual dedicada
<a href="#">EDR-20013</a>	Usar HTMLSlides para materializar Introducing IASI
<a href="#">EDR-20014</a>	Mantener iasi-agent como proyecto separado de IASI
<a href="#">EDR-20015</a>	Usar OpenCode como base inicial de iasi-agent
<a href="#">EDR-20016</a>	Declarar cada proyecto IASI mediante _iasi.yml
<a href="#">EDR-20017</a>	Separar proyecto IASI, repositorio Git y release
<a href="#">EDR-20018</a>	Usar Ubuntu Server 24.04 LTS en iasi-tools
<a href="#">EDR-20019</a>	Usar Ubuntu Server 24.04 LTS en iasi-ai

## **37 Mantener iasi-edr como repositorio independiente**

### **37.1 Contexto**

Las decisiones afectan a múltiples libros y productos. Si viven dentro de uno de ellos, ese repositorio adquiere una autoridad que no corresponde a decisiones transversales.

### **37.2 Decisión**

iasi-edr será un repositorio independiente del Book, Journey y demás productos de IASI.

### **37.3 Alternativas consideradas**

- Integrar los EDR en el repositorio principal de un libro.
- Mantener un directorio ADR dentro de cada producto.

### **37.4 Consecuencias**

El registro de decisiones tiene ciclo de vida y publicación propios. Cualquier repositorio puede referenciarlo sin dependencia jerárquica.

## 38 Publicar iasi-edr como Quarto book estructurado

### 38.1 Contexto

Los EDR son documentos individuales organizados por áreas, pero también forman una publicación consultable como conjunto. Una web plana perdería la relación natural entre área y documentos.

### 38.2 Decisión

`iasi-edr` será un proyecto Quarto de tipo book y utilizará la estrategia de publicación `structured` de `iasi.quarto`.

### 38.3 Alternativas consideradas

- Publicarlo como website.
- Mantener solo archivos QMD sin publicación conjunta.
- Construir manualmente el `book.chapters`.

### 38.4 Consecuencias

Cada área puede representarse como una parte del libro y cada EDR como documento de esa estructura, mientras `iasi.quarto` puede derivar el `_book-structure.yml`.

## 39 Definir las áreas EDR en `_iasi.yml`

### 39.1 Contexto

El generador de índices, el futuro Addin y cualquier validación necesitan conocer la correspondencia entre código, nombre y directorio de cada área. Duplicar esa información en scripts y páginas produciría divergencia.

### 39.2 Decisión

`_iasi.yml` será la fuente estructural legible por máquina para el catálogo de áreas EDR, incluyendo como mínimo `id`, `name` y `path`. `index.qmd` permanece como contenido humano y no se convierte en un contenedor de configuración oculto.

### 39.3 Alternativas consideradas

- Codificar las áreas directamente en R.
- Mantener la tabla únicamente en `index.qmd`.
- Crear un catálogo duplicado por cada consumidor.

### 39.4 Consecuencias

Los consumidores comparten una única definición de estructura y el contenido editorial sigue separado de la configuración.

## 40 Organizar físicamente los EDR por área

### 40.1 Contexto

La clasificación del identificador debe reflejarse también en la organización física para que el repositorio sea comprensible sin necesidad de ejecutar herramientas.

### 40.2 Decisión

Los documentos vivirán bajo `edr/<nn-area>/`. La estructura estable incluye los directorios 00-general, 10-functional, 20-architecture, 30-constraints, 40-data, 50-integration, 60-security, 70-quality, 80-operation y 99-pending.

### 40.3 Alternativas consideradas

- Guardar todos los EDR en un único directorio.
- Separarlos por producto afectado.

### 40.4 Consecuencias

La ruta física coincide con la clasificación primaria. Un documento tiene una sola ubicación principal, mientras que sus relaciones transversales se expresan mediante metadatos.

# 41 Regenerar los índices EDR en pre-render

## 41.1 Contexto

Los índices de área son contenido derivado y deben estar actualizados cuando se publica, sin exigir una acción manual adicional después de crear o modificar un EDR.

## 41.2 Decisión

Quarto ejecutará `R/create-index.R` mediante `project.pre-render`. El script leerá las áreas desde `_iasi.yml`, recorrerá los EDR de cada directorio, extraerá `id` y `title`, validará la correspondencia con el área y regenerará el `index.qmd` correspondiente.

## 41.3 Alternativas consideradas

- Actualizar índices desde el Addin únicamente.
- Regenerarlos mediante una tarea manual separada.
- Integrar la lógica directamente en `iasi.quarto`.

## 41.4 Consecuencias

El render siempre parte de índices frescos. La publicación falla si la estructura física contradice la configuración, evitando resultados silenciosamente incompletos.

## 42 Mantener la lógica específica de EDR fuera de `iasi.quarto`

### 42.1 Contexto

`iasi.quarto` proporciona infraestructura genérica de publicación. La generación de catálogos por áreas EDR responde a un modelo particular que, por ahora, no ha demostrado ser necesario en otras publicaciones.

### 42.2 Decisión

La lógica específica de EDR permanecerá en `iasi-edr`, inicialmente bajo `R/`. Solo se promoverá a `iasi.quarto` cuando exista evidencia de reutilización real.

### 42.3 Alternativas consideradas

- Incorporar inmediatamente cualquier automatización nueva a `iasi.quarto`.
- Duplicar la lógica entre ambos repositorios.

### 42.4 Consecuencias

`iasi.quarto` conserva una responsabilidad genérica y EDR puede evolucionar con rapidez sin convertir una necesidad local en contrato global.

## 43 Separar la configuración común de los perfiles HTML y PDF

### 43.1 Contexto

HTML y PDF comparten estructura documental pero requieren presentación, recursos y directorios de salida distintos. Mezclar ambas configuraciones en un único YAML hace difícil distinguir qué es común y qué depende del formato.

### 43.2 Decisión

`_quarto.yml` contendrá la configuración común y declarará el grupo de perfiles. `_quarto-html.yml` y `_quarto-pdf.yml` contendrán respectivamente la configuración específica de HTML y PDF.

### 43.3 Alternativas consideradas

- Un único `_quarto.yml` con condicionales o configuración acumulada.
- Repositorios separados para cada formato.

### 43.4 Consecuencias

Cada perfil puede evolucionar sin contaminar al otro y la configuración común permanece visible en un único lugar.

## 44 Usar la navbar como navegación principal de iasi-edr en HTML

### 44.1 Contexto

Un Quarto Book ofrece por defecto sidebar de libro y TOC de página. En EDR esos dos elementos duplican la navegación porque las áreas ya forman un catálogo pequeño y estable accesible desde la cabecera.

### 44.2 Decisión

En el perfil HTML de *iasi-edr* la navegación principal será la navbar con Inicio y las áreas EDR. La sidebar izquierda se desactiva y el TOC flotante derecho también se desactiva.

### 44.3 Alternativas consideradas

- Mantener la navegación estándar de Quarto Book.
- Usar solo sidebar sin navbar.

### 44.4 Consecuencias

El contenido gana anchura y el usuario dispone de un único mapa de navegación. Esta decisión afecta solo a HTML y no se incorpora a la configuración común del proyecto.

## 45 Componer el estilo HTML con SCSS común y específico

### 45.1 Contexto

El ecosistema necesita identidad visual compartida, pero cada producto puede requerir reglas propias. Un solo fichero global haría que ajustes locales terminasen afectando a publicaciones no relacionadas.

### 45.2 Decisión

Los estilos vivirán en `resources/css/`. `iasi.scss` contendrá la base visual común y `edr.scss` las especializaciones de EDR. El theme HTML cargará primero el estilo común y después el específico.

### 45.3 Alternativas consideradas

- Mantener CSS independiente y duplicado en cada producto.
- Incluir todas las reglas EDR en el fichero común.

### 45.4 Consecuencias

EDR puede sobrescribir detalles sin romper la identidad general. El orden de carga deja explícita la relación base-especialización.

## **46 Mantener iasi-graphics deliberadamente simple**

### **46.1 Contexto**

La mayoría de las necesidades repetibles de IASI son diagramas editoriales sencillos: cajas, círculos, relaciones y composiciones previsibles. Los casos visuales complejos son mucho menos frecuentes y necesitan iteración específica.

### **46.2 Decisión**

`iasi-graphics` se limitará en su fase actual a generar gráficos sencillos, claros y correctos. Las figuras complejas se resolverán excepcionalmente con herramientas de generación de imagen, PowerPoint u otra herramienta adecuada al caso.

### **46.3 Alternativas consideradas**

- Construir desde el inicio un motor gráfico general.
- Resolver manualmente incluso los patrones sencillos repetidos.

### **46.4 Consecuencias**

La herramienta permanece pequeña y automatiza justo la parte repetible. Los casos raros no dictan la arquitectura del caso común.

## **47 Crear un espacio dedicado para capacidades agentic de IASI**

### **47.1 Contexto**

A medida que IASI incorpora comandos, skills, agentes y adaptadores, esas capacidades dejan de ser simples notas de uso de una herramienta concreta y forman una capa reutilizable del ecosistema.

### **47.2 Decisión**

IASI mantendrá un espacio dedicado **agentics** para organizar comandos, skills, adaptadores y demás capacidades agentic que surjan de la metodología, separándolas de los libros y de la implementación de productos concretos.

### **47.3 Alternativas consideradas**

- Guardar cada skill dentro del repositorio donde se utilizó por primera vez.
- Acoplar las capacidades a una única plataforma de agentes.

### **47.4 Consecuencias**

La metodología puede definir capacidades propias y adaptar su ejecución a plataformas como Codex sin convertir la plataforma en la definición del comportamiento.

## 48 Separar el entorno de laboratorios mediante una máquina virtual dedicada

### 48.1 Contexto

Los primeros laboratorios pueden realizarse prácticamente desde cualquier entorno, pero a medida que avanzan empiezan a necesitar herramientas, runtimes, servicios, contenedores y productos propios de IASI.

Utilizar para ellos la misma máquina empleada para desarrollar IASI mezclaría dos contextos diferentes. El entorno de desarrollo contiene repositorios fuente, herramientas internas y configuraciones que un usuario de los labs no debería necesitar.

Además, utilizar un entorno separado permite comprobar que los productos IASI funcionan fuera de la máquina en la que fueron desarrollados.

### 48.2 Decisión

Los laboratorios que requieran una infraestructura específica utilizarán una **máquina virtual dedicada a labs**, separada del entorno `iasi-dev`.

El Lab 10 del Volumen II será responsable de construir ese entorno de referencia. Los laboratorios posteriores podrán asumir la infraestructura creada a partir de ese punto.

La máquina podrá conservar estados reproducibles mediante checkpoints cuando resulte útil para la ejecución o repetición de los laboratorios.

### 48.3 Alternativas consideradas

- Utilizar directamente la máquina de desarrollo `iasi-dev`.
- Instalar las dependencias de los labs sobre la máquina habitual del usuario.
- Crear un entorno completamente automatizado antes de realizar manualmente los laboratorios de infraestructura.

## 48.4 Consecuencias

El entorno donde se construye IASI queda separado del entorno donde se utiliza IASI.

Los laboratorios permiten comprobar que los productos pueden instalarse y utilizarse sin depender de sus repositorios fuente.

La máquina virtual proporciona aislamiento y permite conservar estados conocidos para repetir experimentos o recuperar el entorno.

La infraestructura de labs pasa a ser un elemento explícito que deberá mantenerse y evolucionar junto con los propios laboratorios.

## 49 Usar HTMLSlides para materializar Introducing IASI

### 49.1 Contexto

Una vez identificada la necesidad de crear una presentación de IASI apareció inmediatamente la cuestión de qué herramienta utilizar para materializarla.

La primera reacción fue asumir **PowerPoint** y buscar la mejor forma de producir la presentación con esa tecnología. A partir de ahí se consideraron también otras herramientas de presentación.

Ese enfoque reveló un problema metodológico: se estaba seleccionando una solución antes de terminar de definir la necesidad.

Esto contradecía un principio fundamental de IASI:

No partir de una herramienta o metodología y preguntar cómo resolver el problema con ella. Partir de la necesidad y evaluar después qué solución la satisface mejor.

Se volvió por tanto a la necesidad.

La presentación debía:

- seguir siendo una **presentación**, no convertirse en un sitio web ni en documentación;
- permitir una narrativa progresiva y controlada;
- ser extremadamente limpia y visual;
- permitir interacción y revelado progresivo cuando resulte útil;
- ofrecer control completo sobre el lenguaje visual de IASI;
- poder mantenerse como un artefacto textual, versionable y reproducible;
- poder ser leído y modificado por agentes dentro del workspace;
- evitar que la identidad visual quede determinada por la herramienta utilizada;
- poder ejecutarse y distribuirse con la menor dependencia posible de un entorno de autoría específico.

Con estos criterios se reevaluaron las alternativas.

## 49.2 Decisión

La presentación **Introducing IASI** se materializará inicialmente mediante **HTMLSlides**.

HTMLSlides se utilizará como infraestructura de presentación y renderer, no como definición del lenguaje visual de IASI.

El diseño, la tipografía, la composición, los diagramas y el resto de decisiones visuales pertenecen a IASI y deberán poder expresarse posteriormente mediante otros renderers cuando sea necesario.

La salida principal será una presentación HTML ejecutable en navegador y mantenida como artefacto versionable dentro del proyecto.

## 49.3 Alternativas consideradas

### 49.3.1 PowerPoint

#### Ventajas

- Herramienta ampliamente conocida y utilizada.
- Muy buen soporte para edición manual de slides.
- Flujo de presentación maduro.
- Amplias capacidades gráficas, de animación y composición.
- Buena interoperabilidad en contextos corporativos.

#### Inconvenientes

- El formato de autoría no es especialmente adecuado para inspección y edición textual.
- El control de versiones y la revisión de cambios son menos naturales que sobre fuentes de texto.
- La automatización y modificación por agentes introduce una capa adicional de herramientas.
- Favorece un flujo de trabajo centrado en la herramienta de edición.
- La decisión inicial de usar PowerPoint se había tomado antes de definir correctamente la necesidad.

#### Resultado

Descartado como tecnología principal para esta presentación. Puede seguir siendo útil para materializaciones o casos concretos futuros.

### 49.3.2 Gamma

#### Ventajas

- Generación rápida de presentaciones con asistencia de IA.
- Buenos resultados visuales iniciales con poco trabajo.
- Publicación y compartición web sencillas.
- Reduce considerablemente el esfuerzo de composición inicial.

#### Inconvenientes

- Introduce dependencia de un servicio externo y de su flujo de trabajo.
- Parte de la lógica de autoría y presentación queda controlada por la plataforma.
- Menor control sobre una fuente canónica completamente gobernada por IASI.
- La reproducibilidad y evolución mediante el workspace son menos directas.
- Sus decisiones visuales y de composición pueden condicionar el lenguaje visual de IASI.

#### Resultado

Descartado porque optimiza la generación de presentaciones, pero proporciona menos control sobre el artefacto y su proceso de ingeniería.

### 49.3.3 Prezi

#### Ventajas

- Narrativa visual diferenciada.
- Permite relaciones espaciales y navegación no lineal.
- Puede producir presentaciones visualmente muy expresivas.

#### Inconvenientes

- Su modelo de navegación condiciona fuertemente la narrativa.
- El movimiento y la estructura espacial no son una necesidad de Introducing IASI.
- Introduce dependencia de una plataforma y un formato de autoría específicos.
- Resulta menos natural como artefacto textual y versionable dentro del workspace.

#### Resultado

Descartado porque su principal ventaja no responde a una necesidad identificada y añadiría una forma narrativa impuesta por la herramienta.

#### **49.3.4 HTML desarrollado específicamente para la presentación**

##### **Ventajas**

- Control completo sobre presentación, comportamiento y lenguaje visual.
- Fuente textual y directamente versionable.
- Ejecución universal mediante navegador.
- Fácil integración con agentes y herramientas del workspace.
- Posibilidad de generar un único artefacto HTML autocontenido.

##### **Inconvenientes**

- Obliga a construir y mantener desde cero comportamiento propio de una presentación.
- Navegación, ajuste al viewport, teclado, notas, progresión y otras capacidades básicas pasarían a ser responsabilidad del proyecto.
- Introduciría infraestructura propia antes de demostrar que realmente es necesaria.

##### **Resultado**

La aproximación HTML se considera adecuada, pero se evita construir desde cero la infraestructura que ya puede proporcionar una solución especializada.

#### **49.3.5 HTMLSlides**

##### **Ventajas**

- Mantiene HTML como fuente y como materialización natural.
- Permite trabajar con archivos textuales, versionables e inspeccionables.
- Puede ser leído y modificado directamente por agentes.
- Proporciona una estructura específica de slides y capacidades básicas de presentación.
- Se ejecuta directamente en navegador.
- Permite producir una presentación autocontenida con muy pocas dependencias.
- Conserva un alto grado de control sobre HTML, CSS y comportamiento.
- No obliga a adoptar una identidad visual concreta.
- Permite que el lenguaje visual de IASI se defina independientemente del renderer.

##### **Inconvenientes**

- Tiene un ecosistema y una madurez menores que PowerPoint.
- Requiere trabajo propio de diseño y CSS.
- Algunas capacidades avanzadas deberán implementarse o adaptarse cuando aparezca la necesidad.
- La interoperabilidad directa con formatos tradicionales de presentación no es su objetivo principal.

## Resultado

Seleccionado porque proporciona suficiente infraestructura de presentación sin apropiarse del diseño ni del proceso de ingeniería.

## 49.4 Consecuencias

`introducing-iasi` tendrá como materialización principal una presentación HTML basada en HTMLSlides.

El contenido, la narrativa y el lenguaje visual permanecen bajo control de IASI.

HTMLSlides se considera un renderer sustituible. La presentación no debe diseñarse conceptualmente alrededor de las limitaciones o convenciones propias de HTMLSlides.

La elección permite mantener la presentación como artefacto textual, reproducible, versionable y accesible a agentes dentro del workspace.

La decisión podrá revisarse si las necesidades reales de la presentación dejan de estar bien cubiertas por HTMLSlides. La herramienta es consecuencia de la necesidad, no una restricción impuesta al problema.

# 50 Mantener iasi-agent como proyecto separado de IASI

## 50.1 Contexto

IASI ya dispone de un runtime y de comandos destinados a soportar la aplicación de la metodología.

Por otra parte, `iasi-dev` es el entorno utilizado para desarrollar IASI.

La aparición de la necesidad de experimentar con un agente local capaz de trabajar sobre el workspace no implica que esa implementación deba incorporarse directamente al runtime de IASI.

Todavía se están descubriendo las capacidades que debe tener un agente IASI, la forma de conectarlo con modelos locales, la relación con skills, comandos y herramientas, y el grado de adaptación necesario sobre plataformas agentic existentes.

Introducir esa experimentación directamente dentro de IASI mezclaría una capacidad todavía en evolución con el núcleo que define y ejecuta la metodología.

## 50.2 Decisión

Se mantendrá `iasi-agent` como un proyecto separado de IASI.

`iasi-agent` será el espacio donde experimentar, integrar y validar una implementación de agente capaz de ejecutar trabajo IASI sobre un workspace.

La separación conceptual será:

```
IASI
↓
define el trabajo, reglas y workflow

iasi-agent
↓
ejecuta trabajo IASI como uno de sus posibles agentes
```

IASI no dependerá de `iasi-agent` para definir la metodología.

`iasi-agent` dependerá de los contratos y artefactos que IASI establezca para poder ejecutar trabajo correctamente.

El nombre `iasi-agent` identifica esta línea de trabajo y puede evolucionar si la comprensión posterior del producto lo requiere.

## 50.3 Alternativas consideradas

### 50.3.1 Incorporar el agente directamente dentro de IASI

Reduciría inicialmente el número de proyectos, pero acoplaría el runtime de la metodología con una implementación agentic concreta todavía experimental.

También dificultaría distinguir qué comportamiento pertenece a IASI y qué comportamiento pertenece al ejecutor.

### 50.3.2 Tratar el agente como parte de `iasi-dev`

`iasi-dev` es un entorno de desarrollo, no un producto de ejecución IASI.

Ubicar allí el agente confundiría infraestructura de desarrollo con una capacidad reutilizable del ecosistema.

### 50.3.3 Utilizar únicamente adaptadores para agentes externos

Los adaptadores siguen siendo necesarios para proyectar capacidades IASI sobre otras plataformas.

Sin embargo, no sustituyen la necesidad de disponer de un espacio propio donde experimentar con un agente IASI y con ejecución local.

## 50.4 Consecuencias

IASI conserva un núcleo independiente de cualquier implementación agentic concreta.

`iasi-agent` puede evolucionar rápidamente, cambiar de plataforma base o incluso descartarse sin obligar a rediseñar el runtime de IASI.

La experimentación local queda aislada del núcleo metodológico.

La decisión se relaciona con EDR-20011, que ya estableció que las capacidades agentic de IASI no deben quedar acopladas a una única plataforma.

La separación permite comprobar en la práctica qué necesita realmente un agente IASI antes de convertir esas necesidades en contratos o capacidades canónicas del sistema.

# 51 Usar OpenCode como base inicial de iasi-agent

## 51.1 Contexto

Una vez definida la necesidad de disponer de un agente capaz de trabajar directamente sobre un workspace IASI, la siguiente cuestión fue determinar si debía construirse desde cero o apoyarse sobre una plataforma agentic existente.

La selección no debía comenzar por la herramienta.

Primero se identificaron las capacidades necesarias:

- trabajar desde el workspace real;
- leer y modificar archivos de forma controlada;
- ejecutar comandos;
- utilizar modelos locales servidos por Ollama;
- no quedar ligado a un único proveedor de modelos;
- permitir reglas e instrucciones de proyecto;
- incorporar herramientas externas;
- soportar MCP;
- permitir capacidades reutilizables como skills y comandos;
- ofrecer control sobre permisos;
- poder utilizarse desde terminal;
- permitir posteriormente integración programática;
- mantener separadas la metodología IASI y la plataforma de ejecución.

A partir de estas necesidades se evaluaron varias alternativas.

## 51.2 Decisión

`iasi-agent` utilizará inicialmente **OpenCode** como plataforma base de ejecución agentic.

OpenCode se considera infraestructura sustituible.

IASI seguirá siendo propietario de las tareas, reglas, skills, contratos y workflow que definan el comportamiento esperado. OpenCode proporciona las capacidades de ejecución necesarias para que un modelo pueda operar sobre el workspace.

Conceptualmente:

```
IASI
↓
tasks / rules / skills / workflow
↓
adapter / configuration
↓
OpenCode
↓
model provider
↓
Ollama / cloud / other
```

La elección de OpenCode no convierte sus convenciones en el modelo conceptual de IASI.

Cuando una capacidad IASI necesite proyectarse al formato esperado por OpenCode, esa traducción deberá mantenerse como una adaptación de plataforma.

## 51.3 Alternativas consideradas

### 51.3.1 Ollama + Open WebUI

#### Ventajas

- Infraestructura local ya disponible.
- Sencillez de instalación y operación.
- Independencia de proveedores cloud para la inferencia.
- Adecuado para conversación, análisis y uso interactivo de modelos.

#### Inconvenientes

- Ollama es un runtime de modelos, no un agente de workspace.
- Open WebUI proporciona principalmente una interfaz de interacción con los modelos.
- No resuelve por sí mismo la ejecución controlada de tareas sobre archivos, terminal y herramientas del proyecto.

## Resultado

Se mantiene como infraestructura de modelos e interfaz de conversación, pero no como base suficiente de `ias-agent`.

### 51.3.2 Continue

#### Ventajas

- Dispone de modo Agent con lectura, edición de archivos y ejecución de comandos.
- Puede utilizar Ollama y modelos locales.
- Ofrece herramientas integradas y soporte para MCP.
- Integra bien el contexto del código y del workspace desde el IDE.
- Permite reglas y políticas de herramientas.

#### Inconvenientes

- Su experiencia principal está integrada en IDEs.
- Vincular `ias-agent` a Continue haría que la capacidad agentic quedase demasiado asociada a una extensión y a un entorno de desarrollo concreto.
- IASI necesita que el agente pueda existir independientemente del editor utilizado.

## Resultado

Alternativa válida para utilizar IASI desde un IDE, pero no seleccionada como base principal de `ias-agent`.

### 51.3.3 Aider

#### Ventajas

- Herramienta madura orientada a terminal.
- Excelente integración con repositorios Git.
- Puede trabajar con modelos locales mediante Ollama.
- Dispone de repo map para proporcionar contexto del repositorio.
- Ofrece un flujo sólido de edición, diff, commit y undo.
- Puede editar también documentación y archivos de configuración, no únicamente código.

#### Inconvenientes

- Su modelo está especialmente orientado a pair programming y edición de repositorios Git.
- El flujo de trabajo gira alrededor de conversaciones de edición, selección de archivos y commits.

- Encaja peor como plataforma general sobre la que proyectar las distintas capacidades agentic de IASI.
- La extensión mediante skills, agentes, permisos y herramientas no constituye el centro de su modelo del mismo modo que en OpenCode.

## Resultado

Muy buena alternativa para edición asistida de código, pero menos adecuada como sustrato general de un agente IASI.

### 51.3.4 Codex CLI

#### Ventajas

- Agente de terminal maduro capaz de leer, modificar y ejecutar trabajo sobre el workspace.
- Ya ha sido utilizado en proyectos IASI.
- Dispone de controles de aprobación y sandbox.
- Puede utilizar modelos OpenAI y también modelos locales mediante Ollama.
- Demuestra en la práctica que una tarea IASI puede ser ejecutada por un agente independiente.

#### Inconvenientes

- Es una plataforma agentic definida y evolucionada dentro del ecosistema OpenAI.
- Utilizarla como identidad de `iasi-agent` acercaría la implementación propia de IASI a una plataforma concreta.
- El uso mediante servicios y modelos OpenAI puede estar sujeto a disponibilidad y cuotas.
- IASI quiere conservar la posibilidad de experimentar sobre una base deliberadamente neutral respecto al proveedor.

## Resultado

Se mantiene como ejecutor válido de tareas IASI y como referencia funcional, pero no se adopta como identidad ni base principal de `iasi-agent`.

### 51.3.5 Construir un agente desde cero

#### Ventajas

- Control completo de arquitectura, comportamiento y contratos.
- Ninguna dependencia conceptual de una plataforma agentic existente.
- Posibilidad de implementar exclusivamente las capacidades necesarias para IASI.

#### Inconvenientes

- Obliga a reconstruir capacidades ya disponibles: sesiones, tool calling, edición, shell, permisos, modelos, MCP, interfaces y gestión del contexto.
- Aumenta drásticamente el trabajo antes de haber descubierto qué partes necesitan realmente una implementación propia.
- Violenta el criterio de no construir una solución nueva cuando una infraestructura existente satisface la necesidad.

## Resultado

Descartado como punto de partida. Podrán sustituirse componentes concretos cuando exista una razón de ingeniería para hacerlo.

### 51.3.6 OpenCode

#### Ventajas

- Es un agente de código abierto y orientado al workspace.
- Puede utilizar múltiples proveedores y modelos locales mediante Ollama.
- Dispone de herramientas para operar sobre el código base y permite extenderlas con herramientas propias.
- Soporta servidores MCP locales y remotos.
- Permite definir y controlar permisos de herramientas.
- Soporta agentes configurables.
- Soporta instrucciones de proyecto.
- Permite comandos personalizados definidos en Markdown.
- Soporta skills mediante archivos `SKILL.md`.
- Puede utilizarse desde terminal sin depender de un IDE.
- Ofrece ejecución headless mediante servidor HTTP.
- Dispone de SDK para integración programática.
- Su arquitectura permite separar cliente, servidor, modelo y workspace.

#### Inconvenientes

- Sigue siendo una dependencia externa que puede evolucionar.
- Sus convenciones (`AGENTS.md`, `.opencode/`, configuración, etc.) no coinciden necesariamente con las abstracciones canónicas de IASI.
- Los modelos locales pequeños pueden limitar la calidad del tool calling y del razonamiento agentic.
- Será necesario mantener una capa de adaptación para evitar que IASI dependa de formatos específicos de OpenCode.

## Resultado

Seleccionado como base inicial porque proporciona la mayor parte de la infraestructura necesaria sin obligar a IASI a construir un agente desde cero ni a ligarse a un proveedor de modelos o a un IDE concreto.

## 51.4 Consecuencias

`iasi-agent` podrá comenzar como una adaptación/configuración de OpenCode en lugar de como un agente implementado completamente desde cero.

Los modelos servidos por Ollama podrán utilizarse como proveedores de inferencia para el agente.

Las reglas, skills y comandos de IASI no deberán definirse canónicamente en formatos propios de OpenCode. Se proyectarán mediante adapters cuando sea necesario.

La integración con OpenCode deberá respetar las reglas de permisos e inmutabilidad establecidas por IASI.

OpenCode se considera sustituible.

Si en el futuro deja de satisfacer las necesidades de IASI, podrá reemplazarse sin cambiar la definición de las tareas ni el workflow.

La decisión sigue el mismo criterio aplicado en otras partes del ecosistema:

**Utilizar una herramienta para lo que resuelve bien sin convertirla en la definición del problema.**

## **52 Declarar cada proyecto IASl mediante \_iasi.yml**

## 53 Declarar cada proyecto IASI mediante `_iasi.yml`

### 53.1 Contexto

Durante la reorganización de repositorios surgió la necesidad de distinguir entre un directorio que simplemente contiene archivos y un proyecto que IASI debe reconocer como unidad de ingeniería.

También existía el directorio `.iasi/`, destinado a material de trabajo interno, lo que podía inducir a utilizarlo como marcador de identidad del proyecto.

Sin embargo, el estado interno y la declaración persistente cumplen responsabilidades diferentes.

### 53.2 Decisión

Todo proyecto IASI tendrá una declaración explícita mediante un archivo `_iasi.yml` situado en la raíz del proyecto.

`_iasi.yml` define la identidad y configuración declarativa del proyecto.

El directorio `.iasi/`, cuando exista, se reservará para estado interno o material de trabajo que pueda recrearse, descartarse o regenerarse.

Por tanto:

<code>_iasi.yml</code>	→ declaración persistente del proyecto
<code>.iasi/</code>	→ estado y material interno de trabajo

Un repositorio que actúe además como ámbito raíz de varios proyectos podrá ser también un proyecto IASI y disponer de su propia declaración raíz.

Los proyectos hijos mantienen sus propias declaraciones y su identidad independiente.

## 53.3 Rationale

La identidad de un proyecto no debe depender de la existencia de cachés, índices, estado temporal u otros materiales internos.

Una declaración explícita permite descubrir proyectos, validar configuración y combinar información de distintos ámbitos sin confundir identidad con estado de ejecución.

## 53.4 Alternativas consideradas

### 53.4.1 Utilizar `.iasi/` como marcador de proyecto

Se descarta porque el directorio representa estado interno y puede ser regenerable o prescindible.

### 53.4.2 Inferir que existe un proyecto por la presencia de archivos de una tecnología concreta

Se descarta porque un proyecto IASI no debe depender de Quarto, R, Java, Git u otra tecnología particular para ser reconocido.

## 53.5 Consecuencias

Las herramientas IASI podrán descubrir proyectos buscando `_iasi.yml`.

Un repositorio podrá contener proyectos IASI y otros elementos que no lo sean sin que la mera estructura de carpetas determine su identidad.

Las configuraciones raíz y locales podrán combinarse cuando una operación necesite conocer ambos ámbitos.

## **54 Separar proyecto IASI, repositorio Git y release**

## 55 Separar proyecto IASI, repositorio Git y release

### 55.1 Contexto

Un repositorio puede contener varios proyectos IASI: documentación, software y otros elementos independientes. Durante la implementación de **release** apareció la tentación de hacer que cada proyecto hijo conociese el repositorio que lo contiene y decidiese directamente dónde liberar sus artefactos.

Esto mezcla tres ámbitos distintos:

- el proyecto IASI;
- el repositorio Git;
- la colección liberada por el repositorio.

### 55.2 Decisión

IASI distinguirá explícitamente esos tres ámbitos.

Un **proyecto IASI** conoce su propia declaración, fuentes, outputs y operaciones locales. No necesita conocer qué repositorio Git lo contiene.

El **repositorio Git** establece el ámbito de liberación conjunta. Una operación **release** deberá localizar la raíz Git correspondiente y operar desde ese ámbito.

La **release** pertenece al repositorio y reúne los productos o proyectos liberables que éste publica como colección.

Conceptualmente:

```
repositorio Git
 _iasi.yml
 docs/
 user-guide/
 _iasi.yml
 software/
```

```
producto/
 _iasi.yml
release/
```

Si un proyecto hijo constituye a su vez un repositorio Git independiente, ese límite Git define un nuevo ámbito de release.

## 55.3 Configuración

Cuando una operación necesite información del proyecto local y del repositorio que lo contiene, podrá combinar la configuración de ambos ámbitos.

La configuración local describe el proyecto. La configuración raíz describe decisiones de ámbito común, como localizaciones o convenciones de release.

La precedencia exacta podrá implementarse por capas, manteniendo la regla de que la configuración más específica pueda especializar los valores generales.

## 55.4 Rationale

Un proyecto debe poder moverse entre repositorios o existir de forma independiente sin cambiar su identidad interna por razones de control de versiones.

Al mismo tiempo, la release necesita conocer la colección completa que un repositorio publica y no únicamente el proyecto desde el que se lanzó una operación.

## 55.5 Alternativas consideradas

### 55.5.1 Hacer que cada proyecto hijo conozca su repositorio

Se descarta porque acopla el proyecto a una ubicación y a una tecnología de control de versiones concreta.

### 55.5.2 Tratar release como operación puramente local del proyecto

Se descarta porque no permite ensamblar de forma natural una publicación compuesta por varios proyectos del mismo repositorio.

## 55.6 Consecuencias

`release` deberá descubrir la raíz Git de forma robusta antes de ensamblar la colección liberada.

La estructura de `release/` será orientada a productos liberados y no una copia literal de las categorías internas del repositorio.

Por ejemplo:

```
release/
 index.html
 user-guide/
 producto/
```

La raíz de la release podrá incorporar artefactos propios de la colección, como un `index.html`, aunque esos artefactos no pertenezcan a ningún proyecto hijo concreto.

## **56 Usar Ubuntu Server 24.04 LTS en iasi-tools**

# 57 Usar Ubuntu Server 24.04 LTS en iasi-tools

## 57.1 Contexto

La infraestructura estable de IASI separa el entorno de desarrollo interactivo de los servicios y herramientas compartidas. `iasi-tools` será una máquina virtual dedicada a alojar herramientas de ingeniería y servicios auxiliares que deban permanecer disponibles con independencia de `iasi-dev`.

Para este nodo se consideraron principalmente **Debian Server** y **Ubuntu Server**. Ambas distribuciones ofrecen una base Linux adecuada, estable y ampliamente soportada para servicios, contenedores y automatización.

La decisión no debe buscar la distribución más mínima de forma aislada, sino reducir la variabilidad de la infraestructura IASI y simplificar su operación, mantenimiento y evolución.

## 57.2 Decisión

`iasi-tools` utilizará **Ubuntu Server 24.04 LTS** como sistema operativo de referencia.

La instalación será deliberadamente mínima y sin entorno gráfico. Las capacidades necesarias se añadirán sobre esa base según aparezcan necesidades reales de herramientas, servicios o contenedores.

Ubuntu Server 24.04 LTS se considera parte de la infraestructura estable de IASI y no un entorno experimental.

## 57.3 Rationale

Debian proporciona una base excelente para un servidor de herramientas, pero introducirlo únicamente en `iasi-tools` obligaría a mantener una variante adicional de sistema operativo dentro de la infraestructura estable sin aportar una capacidad específica que IASI necesite.

Utilizar Ubuntu Server también en los nodos Linux estables permite compartir criterios de instalación, administración, red, paquetes, Docker, scripts, troubleshooting y futura automatización.

La elección de una versión LTS prioriza estabilidad y duración del soporte frente a incorporar versiones más recientes del sistema operativo sin una necesidad funcional concreta.

El criterio es, por tanto, **minimizar variabilidad accidental**. La diversidad tecnológica se introducirá cuando resuelva un problema real, no como objetivo en sí misma.

## 57.4 Alternativas consideradas

### 57.4.1 Debian Server

Es una alternativa técnicamente válida y especialmente atractiva por su sobriedad y estabilidad.

Se descarta para `iasi-tools` porque sus ventajas no compensan el coste de mantener una segunda distribución Linux estable dentro de IASI.

### 57.4.2 Ubuntu Server 26.04 LTS

Aporta una base más reciente, pero `iasi-tools` no necesita novedades del sistema operativo para cumplir su función.

Se prefiere 24.04 LTS como plataforma madura y conocida. Una migración futura deberá responder a una necesidad o a la evolución normal del ciclo de soporte, no al simple hecho de existir una versión posterior.

### 57.4.3 Utilizar `iasi-dev` para alojar las herramientas

Se descarta porque mezclaría servicios persistentes con el entorno de desarrollo interactivo y volvería a acoplar la disponibilidad de herramientas al estado de la máquina de desarrollo.

## 57.5 Consecuencias

`iasi-tools` se convierte en el servidor Linux estable de herramientas y servicios de ingeniería de IASI.

La infraestructura podrá reutilizar procedimientos comunes con otros nodos Ubuntu Server, especialmente instalación base, red, Docker, mantenimiento y automatización.

Las herramientas que necesiten interfaz gráfica seguirán perteneciendo a entornos interactivos o de laboratorio y no justificarán añadir una GUI al servidor.

Debian permanece como tecnología válida para experimentación o para futuros nodos si aparece una razón arquitectónica concreta para utilizarlo.

## **58 Usar Ubuntu Server 24.04 LTS en iasi-ai**

## 59 Usar Ubuntu Server 24.04 LTS en iasi-ai

### 59.1 Contexto

`iasi-ai` será el nodo estable dedicado al runtime local de Sistemas Inteligentes de IASI. Su responsabilidad incluirá la ejecución de modelos e infraestructura asociada, inicialmente mediante Ollama, y podrá incorporar capacidades de inferencia acelerada y otros servicios relacionados con IA.

Este nodo tiene requisitos diferentes de `iasi-tools`. Además de estabilidad como servidor, debe reducir fricción en componentes especialmente sensibles al sistema operativo y al kernel, como drivers, GPU, runtimes de cómputo, contenedores acelerados y herramientas del ecosistema de IA.

Para el sistema operativo se consideraron principalmente **Debian Server** y **Ubuntu Server**.

### 59.2 Decisión

`iasi-ai` utilizará **Ubuntu Server 24.04 LTS** como sistema operativo de referencia.

La instalación será sin entorno gráfico y orientada a ejecución de servicios. Ollama, modelos y demás runtimes se desplegarán sobre esta base, manteniendo separados el sistema operativo y las capacidades concretas de IA instaladas sobre él.

La posibilidad de utilizar GPU no cambia la identidad del nodo ni obliga a introducir una distribución especializada de IA.

### 59.3 Rationale

En `iasi-ai` la elección de Ubuntu Server no responde únicamente a homogeneizar la infraestructura.

El ecosistema de IA en Linux tiende a ofrecer abundante documentación, paquetes, ejemplos y procedimientos operativos sobre Ubuntu. Esto reduce riesgo y trabajo incidental al integrar drivers, aceleración, contenedores y runtimes de inferencia.

Debian también puede ejecutar estas tecnologías, pero no aporta una ventaja específica para IASI que justifique aceptar más fricción precisamente en el nodo donde la compatibilidad del ecosistema resulta más importante.

Utilizar Ubuntu Server 24.04 LTS aporta además coherencia con `iasi-tools`, permitiendo compartir la base operativa sin mezclar las responsabilidades de ambos servidores.

La elección de 24.04 LTS prioriza una plataforma madura frente a adoptar una versión más reciente únicamente por novedad.

## **59.4 Alternativas consideradas**

### **59.4.1 Debian Server**

Es técnicamente viable y podría proporcionar una base más mínima.

Se descarta porque `iasi-ai` se beneficia especialmente de seguir el camino más habitual del ecosistema Linux de IA y aceleración. La reducción marginal de componentes no compensa una posible mayor fricción de integración y soporte.

### **59.4.2 Ubuntu Server 26.04 LTS**

Puede convertirse en una plataforma adecuada más adelante, pero no existe una necesidad actual que justifique mover el runtime estable de IA a una base más reciente.

La actualización se evaluará cuando aporte una ventaja concreta o cuando el ciclo de vida de 24.04 lo aconseje.

### **59.4.3 Distribución especializada para IA o GPU**

Se descarta como plataforma base porque acoplaría innecesariamente la infraestructura a un stack concreto.

`iasi-ai` debe seguir siendo un servidor Linux general sobre el que puedan cambiar Ollama, modelos, drivers, runtimes o tecnologías de inferencia sin redefinir el nodo completo.

### **59.4.4 Ejecutar el runtime de IA en `iasi-tools`**

Se descarta porque herramientas de ingeniería y ejecución de modelos tienen perfiles de recursos, dependencias y ciclos de cambio diferentes. Mantenerlos separados permite evolucionar, reiniciar, dimensionar o experimentar con el runtime de IA sin afectar a los servicios de herramientas.

## 59.5 Consecuencias

`iasai-ai` constituye un nodo propio de la infraestructura estable y no una extensión de `iasai-tools`.

IASI podrá compartir con `iasai-tools` la base de administración Ubuntu Server, mientras conserva independientes los servicios, dependencias, dimensionamiento y ciclo de vida de cada nodo.

La integración de GPU, si se incorpora, deberá documentarse como capacidad del nodo y no como propiedad esencial del sistema operativo elegido.

Los cambios de runtime de IA podrán producirse sin alterar esta decisión mientras Ubuntu Server siga proporcionando una base adecuada para su ejecución.

# **Part IV**

## **Constraints**

Esta sección reúne los Engineering Decision Records clasificados en el área **Constraints** (30).

ID	Título
<a href="#">EDR-30001</a>	Diseñar las publicaciones IASI para escritorio
<a href="#">EDR-30002</a>	Evitar la generalización prematura
<a href="#">EDR-30003</a>	Construir manualmente la infraestructura antes de automatizarla
<a href="#">EDR-30004</a>	Usar Windows como sistema operativo de referencia para los labs

## **60 Diseñar las publicaciones IASI para escritorio**

### **60.1 Contexto**

El uso principal previsto para las publicaciones IASI es escritorio. Ajustar continuamente layouts, tamaños y componentes para móvil añade trabajo y condiciona una interfaz que se está diseñando y evaluando en pantallas de escritorio.

### **60.2 Decisión**

La referencia de diseño será escritorio. No se añadirán media queries ni ajustes responsive específicos para móvil salvo que una necesidad futura los justifique expresamente.

### **60.3 Alternativas consideradas**

- Diseñar mobile-first.
- Mantener paridad visual obligatoria entre escritorio y móvil desde el inicio.

### **60.4 Consecuencias**

El CSS permanece más pequeño y la revisión visual se hace contra una referencia estable. La ausencia de trabajo móvil es deliberada, no un olvido.

# 61 Evitar la generalización prematura

## 61.1 Contexto

IASI genera patrones que podrían parecer reutilizables desde su primera aparición. Convertir inmediatamente cada patrón en infraestructura compartida crea APIs y dependencias antes de conocer sus verdaderos límites.

## 61.2 Decisión

Una solución permanecerá local mientras solo exista una necesidad demostrada. La promoción a componente común se hará cuando aparezca reutilización real y pueda definirse un contrato estable.

## 61.3 Alternativas consideradas

- Centralizar desde el primer uso.
- Diseñar extensibilidad para escenarios todavía hipotéticos.

## 61.4 Consecuencias

La arquitectura común crece por evidencia. Los productos conservan libertad para experimentar sin comprometer prematuramente a todo el ecosistema.

## **62 Construir manualmente la infraestructura antes de automatizarla**

### **62.1 Contexto**

Automatizar una máquina o entorno que todavía no se comprende convierte supuestos provisionales en código y dificulta distinguir necesidad real de accidente de la primera instalación.

### **62.2 Decisión**

La construcción inicial de la máquina y la infraestructura se hará manualmente. Después de experimentar, aprender, documentar y especificar el entorno, se automatizarán las partes estables.

### **62.3 Alternativas consideradas**

- Empezar directamente con infraestructura totalmente automatizada.
- Mantener permanentemente una instalación manual.

### **62.4 Consecuencias**

La automatización captura conocimiento comprobado en lugar de congelar descubrimientos provisionales. El proceso manual inicial forma parte del aprendizaje.

## **63 Usar Windows como sistema operativo de referencia para los labs**

### **63.1 Contexto**

Los laboratorios necesitan un entorno de referencia concreto para poder describir instalaciones, comandos, rutas, herramientas y procedimientos de manera reproducible.

Intentar documentar desde el principio cada laboratorio simultáneamente para Windows, Linux, macOS y otras combinaciones multiplicaría las variantes y desviaría el contenido del objetivo del propio laboratorio.

IASI no pretende depender de un único sistema operativo, pero los labs necesitan un camino principal que podamos ejecutar, verificar y mantener.

### **63.2 Decisión**

**Windows será el sistema operativo de referencia por defecto para los laboratorios de IASI.**

Las instrucciones principales de los labs se escribirán y verificarán sobre Windows.

Esto no implica que IASI sea un proyecto exclusivo de Windows ni que los laboratorios no puedan realizarse sobre otros sistemas operativos. Otros entornos podrán utilizarse cuando sea posible, pero no será obligatorio mantener una variante equivalente de cada paso.

### **63.3 Alternativas consideradas**

- Utilizar Linux como sistema operativo de referencia.
- Mantener instrucciones equivalentes para varios sistemas operativos desde el inicio.
- No establecer ningún sistema operativo de referencia.

## 63.4 Consecuencias

Los laboratorios disponen de una plataforma base conocida sobre la que pueden escribirse instrucciones reproducibles.

Se reduce la complejidad editorial y de validación al evitar una matriz de procedimientos por sistema operativo.

La metodología y los productos IASI deberán continuar evitando dependencias innecesarias de Windows cuando no formen parte de una necesidad real.

La compatibilidad con otros sistemas podrá desarrollarse y validarse posteriormente sin condicionar el diseño inicial de cada lab.

**Part V**

**Datos**

Esta sección reúne los Engineering Decision Records clasificados en el área **Datos** (40).

ID	Título
<a href="#">EDR-40001</a>	Usar el front matter como fuente de metadatos del EDR
<a href="#">EDR-40002</a>	Tratar los índices de área como datos derivados
<a href="#">EDR-40003</a>	Hacer los EDR consultables por metadatos y texto completo

## 64 Usar el front matter como fuente de metadatos del EDR

### 64.1 Contexto

Los índices, búsquedas, validaciones y herramientas de creación necesitan leer información estructurada de cada decisión sin interpretar su prosa.

### 64.2 Decisión

Cada EDR almacenará en el front matter, como mínimo, `id`, `title`, `status`, `area`, `created`, `tags`, `affects`, `related`, `supersedes` y `superseded-by`. El cuerpo del documento conserva el razonamiento.

### 64.3 Alternativas consideradas

- Inferir metadatos del nombre del archivo.
- Mantener un catálogo YAML externo con todos los documentos.
- Extraer clasificación mediante análisis del texto.

### 64.4 Consecuencias

Cada documento es autocontenido y legible por máquina. Las herramientas pueden indexarlo sin mantener una segunda base de datos de metadatos.

## **65 Tratar los índices de área como datos derivados**

### **65.1 Contexto**

Un `index.qmd` de área repite identificadores y títulos que ya existen en los EDR. Si se edita como fuente primaria puede quedar desincronizado del conjunto real de documentos.

### **65.2 Decisión**

Los EDR y su front matter son la fuente de verdad. Los `index.qmd` de área se regeneran a partir de ellos y pueden reemplazarse completamente en cada ejecución.

### **65.3 Alternativas consideradas**

- Considerar el índice como catálogo maestro.
- Exigir actualización manual simultánea del EDR y del índice.

### **65.4 Consecuencias**

No hay doble escritura. Un índice puede borrarse y reconstruirse sin pérdida de información.

## **66 Hacer los EDR consultables por metadatos y texto completo**

### **66.1 Contexto**

Clasificar cada decisión en una sola área primaria no basta para responder preguntas transversales como qué decisiones afectan a un producto, cuáles están relacionadas o qué decisiones sustituyen a otras.

### **66.2 Decisión**

El modelo EDR incorporará metadatos de afectación y relaciones, y la publicación debe permitir consulta tanto por esos campos como por el texto completo del documento.

### **66.3 Alternativas consideradas**

- Crear múltiples copias del mismo EDR en cada área afectada.
- Limitar la recuperación a la navegación por carpetas.

### **66.4 Consecuencias**

La clasificación física sigue siendo simple mientras las relaciones transversales se expresan como datos. El registro puede evolucionar hacia búsquedas y vistas derivadas sin cambiar los documentos.

# **Part VI**

## **Integración**

Esta sección reúne los Engineering Decision Records clasificados en el área **Integración** (50).

ID	Título
<a href="#">EDR-50001</a>	Distinguir MCP del servidor MCP
<a href="#">EDR-50002</a>	Exponer PlantUML como capacidad utilizable por modelos
<a href="#">EDR-50003</a>	Clasificar los productos técnicos por su forma de uso

## 67 Distinguir MCP del servidor MCP

### 67.1 Contexto

Usar “MCP” para referirse tanto al protocolo como al software que se instala y configura mezcla dos conceptos distintos y dificulta explicar qué producto ofrece IASI.

### 67.2 Decisión

MCP se utilizará para nombrar el Model Context Protocol. El componente ejecutable que expone capacidades a un modelo se denominará `servidor MCP`.

### 67.3 Alternativas consideradas

- Llamar MCP indistintamente al protocolo y al servidor.
- Inventar un nombre específico de IASI para evitar la terminología estándar.

### 67.4 Consecuencias

La documentación puede explicar con precisión qué estándar se usa y qué componente debe instalarse, configurarse y operarse.

## **68 Exponer PlantUML como capacidad utilizable por modelos**

### **68.1 Contexto**

Generar diagramas PlantUML desde un modelo requiere una frontera clara entre la intención del agente y la herramienta externa que procesa el diagrama.

### **68.2 Decisión**

PlantUML se tratará como una herramienta que puede exponerse mediante un servidor MCP para que un modelo solicite la creación o renderizado de diagramas sin incorporar PlantUML al propio modelo.

### **68.3 Alternativas consideradas**

- Acoplar la generación a una interfaz manual.
- Tratar PlantUML como parte intrínseca del agente.

### **68.4 Consecuencias**

La capacidad puede reutilizarse desde distintos modelos o plataformas y conserva una interfaz explícita de integración.

## 69 Clasificar los productos técnicos por su forma de uso

### 69.1 Contexto

El ecosistema contiene componentes de naturaleza distinta. Llamarlos a todos “herramientas” oculta cómo se consumen y qué debe instalar o configurar el usuario.

### 69.2 Decisión

La documentación distinguirá al menos tres tipos de producto técnico: **Herramienta**, **Extensión** y **Servidor MCP**. Cada producto se presenta según su forma real de integración.

### 69.3 Alternativas consideradas

- Usar “MCP” como categoría de producto.
- Presentar todos los repositorios bajo una única etiqueta genérica.

### 69.4 Consecuencias

La navegación y la documentación pueden comunicar de inmediato el modo de consumo esperado de cada pieza.

## **Part VII**

# **Seguridad**

Esta sección reúne los Engineering Decision Records clasificados en el área **Seguridad (60)**.  
Actualmente no hay EDR registrados en esta área.

## **Part VIII**

# **Calidad**

Esta sección reúne los Engineering Decision Records clasificados en el área **Calidad** (70).

ID	Título
<a href="#">EDR-70001</a>	Validar la estructura Quarto antes de preparar o renderizar
<a href="#">EDR-70002</a>	Fallar ante inconsistencias estructurales del registro EDR
<a href="#">EDR-70003</a>	Regenerar de forma determinista los artefactos derivados
<a href="#">EDR-70004</a>	Preservar el formato deliberado del código Go
<a href="#">EDR-70005</a>	Separar contenido derivable y contenido de autor en artefactos regenerables

## 70 Validar la estructura Quarto antes de preparar o renderizar

### 70.1 Contexto

Un pipeline puede avanzar bastante antes de fallar si faltan archivos esenciales o la configuración no es coherente con el tipo de proyecto.

### 70.2 Decisión

`iasi.quarto` validará al inicio la existencia de `_quarto.yml` y `_iasi.yml`, determinará el tipo de proyecto Quarto y verificará la configuración específica requerida por ese tipo antes de continuar con `prepare/build`. Para publicaciones con perfiles se comprobará la presencia de las configuraciones declaradas.

### 70.3 Alternativas consideradas

- Delegar todos los errores a `quarto render`.
- Validar solo después de haber generado artefactos.

### 70.4 Consecuencias

Los errores de configuración aparecen cerca de su causa y antes de modificar artefactos derivados. El pipeline puede detenerse con mensajes específicos de IASI.

# 71 Fallar ante inconsistencias estructurales del registro EDR

## 71.1 Contexto

Omitir silenciosamente un directorio, un identificador incorrecto o un front matter incompleto permitiría publicar un catálogo aparentemente correcto pero parcial.

## 71.2 Decisión

Las herramientas EDR fallarán ante inconsistencias relevantes: directorios configurados que no existen, documentos sin `id` o `title`, identificadores que no corresponden al área y duplicados detectables.

## 71.3 Alternativas consideradas

- Emitir warnings y continuar siempre.
- Ignorar archivos problemáticos durante la generación del índice.

## 71.4 Consecuencias

Un render fallido es preferible a una publicación incompleta que parezca válida. Los errores estructurales se convierten en defectos visibles y corregibles.

## **72 Regenerar de forma determinista los artefactos derivados**

### **72.1 Contexto**

Los archivos generados introducen incertidumbre si su contenido depende del estado previo o de modificaciones manuales.

### **72.2 Decisión**

Los artefactos derivados se reconstruirán completamente a partir de sus fuentes y con orden estable. En EDR, los índices se ordenarán por identificador y no conservarán contenido manual entre ejecuciones.

### **72.3 Alternativas consideradas**

- Actualizar incrementalmente los índices existentes.
- Preservar secciones manuales dentro de archivos generados.

### **72.4 Consecuencias**

Dos ejecuciones sobre las mismas fuentes producen el mismo resultado. Los cambios generados son revisables y no esconden estado acumulado.

## 73 Preservar el formato deliberado del código Go

### 73.1 Contexto

IASI disponía de una rule específica para Go que exigía que todo el código fuente se ajustase al formato producido por `gofmt` y desaconsejaba mantener manualmente cualquier formato que entrase en conflicto con él.

Durante la revisión del código de `iasi-cli` se comprobó que determinadas decisiones de formato pueden expresar de forma directa la estructura lógica del programa y mejorar su lectura por personas.

Por ejemplo, un `switch` utilizado como tabla de decisión puede escribirse de forma que sus alternativas se comprendan de un vistazo:

```
switch adapter.Action {
 case cli.Help: adapter = helpAdapter(adapter)
 case cli.List: adapter = listAdapters(adapter)
 default: adapter = processAdapter(adapter)
}
```

La indentación muestra que los `case` pertenecen al bloque del `switch`, mientras que la alineación permite comparar visualmente las alternativas y sus acciones.

El formateo automático de Go modifica esta representación aunque el código sea válido, eliminando información visual introducida deliberadamente para facilitar su comprensión.

### 73.2 Decisión

El código Go propiedad de IASI conservará su **formato deliberado**.

No se ejecutarán automáticamente `gofmt`, `go fmt` ni herramientas equivalentes que normalicen el formato del código Go y puedan sustituir decisiones explícitas de indentación, alineación o agrupación visual.

La rule existente que obliga a utilizar `gofmt` se elimina de `iasi-core`.

Las herramientas de desarrollo, automatizaciones y procesos de integración no deberán aplicar formateo automático sobre código Go propiedad del proyecto.

El formato seguirá siendo responsabilidad de la ingeniería del código. Puede modificarse cuando la modificación sea deliberada y mejore o preserve su legibilidad, pero no se normalizará automáticamente por el mero hecho de ajustarse a una convención externa.

## 73.3 Rationale

El formato del código no es únicamente decoración. Puede formar parte de su representación y ayudar a mostrar relaciones, niveles, alternativas y bloques conceptuales.

IASI prioriza que el código pueda **leerse y comprenderse por personas**. Una herramienta automática es útil mientras contribuya a ese objetivo, pero no debe imponerse cuando elimina información visual que el autor ha introducido deliberadamente para expresar mejor la estructura del programa.

La decisión no rechaza las convenciones del ecosistema Go como referencia general. Establece que una convención de formato no tiene prioridad automática sobre una representación deliberada que resulte más clara para el código concreto.

## 73.4 Alternativas consideradas

### 73.4.1 Mantener `gofmt` como obligatorio

Se descarta porque impide conservar formatos deliberados que permiten representar con mayor claridad determinadas estructuras lógicas.

### 73.4.2 Utilizar `gofmt` por defecto y permitir excepciones puntuales

Se descarta porque cualquier ejecución automática puede destruir esas excepciones. Obliga además a introducir mecanismos adicionales para identificar qué bloques deben preservarse.

### 73.4.3 Desactivar únicamente el formateo al guardar

Es insuficiente. El mismo problema puede aparecer mediante `go fmt`, `gofmt`, tareas de build, acciones del editor, agentes o pipelines de integración.

## 73.5 Consecuencias

El código Go de IASI puede utilizar indentación, alineación y espacios de manera deliberada cuando ayuden a expresar su estructura.

Los editores y agentes que trabajen sobre el código no deben ejecutar formateadores automáticos de Go salvo decisión explícita que modifique este EDR.

Las pipelines de build y validación no deben asumir que `gofmt` forma parte de los criterios de aceptación del código.

El formato pasa a revisarse como parte de la legibilidad del código, no como conformidad automática con la salida de una herramienta.

El código generado o mantenido externamente puede conservar el formato impuesto por su generador o por su fuente original cuando no sea propiedad del proyecto.

## 73.6 Principios derivados

**El formato del código puede expresar estructura.**

**La legibilidad para las personas tiene prioridad sobre la normalización automática.**

**El código propiedad de IASI se formatea deliberadamente, no automáticamente.**

## **74 Separar contenido derivable y contenido de autor en artefactos regenerables**

## 75 Separar contenido derivable y contenido de autor en artefactos regenerables

### 75.1 Contexto

Algunos documentos pueden derivarse en gran medida del conocimiento estructurado de un proyecto, pero siguen necesitando aportaciones editoriales humanas: explicaciones, ejemplos, recomendaciones, troubleshooting, Tips & Tricks u otros contenidos que no deben perderse cuando el documento se regenere.

Regenerar el documento completo desde cero destruiría esas aportaciones. Evitar la regeneración impediría mantener actualizada la parte derivable.

### 75.2 Decisión

Los artefactos parcialmente derivables distinguirán explícitamente entre:

- **contenido derivable**, que IASI puede regenerar a partir de fuentes de verdad;
- **contenido de autor**, que se preserva y no se sobrescribe automáticamente.

Cuando la tecnología utilizada lo permita, el contenido de autor se incorporará mediante mecanismos protegidos, como includes o secciones identificadas como autorales.

Al regenerar un artefacto, IASI actualizará únicamente las partes derivables.

Si cambia una dependencia que pueda afectar al contenido de autor, IASI no lo reescribirá automáticamente. Lo marcará como **afectado** o pendiente de revisión.

### 75.3 Rationale

La generación automática y la edición humana no son alternativas excluyentes. Pueden coexistir si el ciclo de vida del artefacto distingue correctamente qué partes son derivables y quién tiene autoridad para modificarlas.

El objetivo no es maximizar el porcentaje de generación automática, sino mantener coherencia sin destruir conocimiento editorial.

## **75.4 Alternativas consideradas**

### **75.4.1 Regenerar siempre el documento completo**

Se descarta porque eliminaría o sobrescribiría aportaciones humanas no derivables.

### **75.4.2 Dejar de regenerar cuando exista contenido manual**

Se descarta porque convertiría una pequeña aportación humana en un bloqueo para mantener actualizado todo el artefacto.

### **75.4.3 Modificar automáticamente el contenido de autor cuando cambien sus dependencias**

Se descarta porque el sistema no debe asumir autoridad sobre contenido deliberadamente protegido.

## **75.5 Consecuencias**

Los generadores deberán conocer los límites entre contenido derivable y contenido de autor.

La trazabilidad de dependencias permitirá detectar cuándo una parte protegida necesita revisión.

Este patrón puede aplicarse a User Guides, Technical Guides y otros artefactos parcialmente materializables, no únicamente a documentación Quarto.

**Regenerar lo derivable; preservar lo autoral; señalar lo afectado.**

# **Part IX**

## **Operación**

Esta sección reúne los Engineering Decision Records clasificados en el área **Operación** (80).

ID	Título
<a href="#">EDR-80001</a>	Usar HTML como perfil por defecto
<a href="#">EDR-80002</a>	Separar los directorios de salida por perfil
<a href="#">EDR-80003</a>	Usar scripts R reutilizables para preview y render
<a href="#">EDR-80004</a>	Separar herramientas principales de trabajo y laboratorios locales
<a href="#">EDR-80005</a>	Postprocesar la salida de Quarto sin forzar su comportamiento

## 76 Usar HTML como perfil por defecto

### 76.1 Contexto

Durante la edición y revisión ordinaria la salida HTML es la más rápida y navegable. PDF sigue siendo una publicación necesaria, pero no necesita activarse implícitamente en cada render.

### 76.2 Decisión

Los proyectos Quarto con perfiles HTML/PDF usarán `html` como perfil por defecto. PDF se solicitará de forma explícita mediante su perfil.

### 76.3 Alternativas consideradas

- No declarar perfil por defecto.
- Renderizar ambos perfiles siempre.
- Usar PDF como perfil predeterminado.

### 76.4 Consecuencias

El comando de render habitual produce HTML sin opciones adicionales y el coste de PDF se paga solo cuando se necesita esa salida.

## 77 Separar los directorios de salida por perfil

### 77.1 Contexto

HTML produce un sitio con múltiples archivos mientras PDF produce un artefacto documental distinto. Mezclar ambas salidas dificulta limpieza, despliegue y revisión.

### 77.2 Decisión

El perfil HTML escribirá en `outputs/html` y el perfil PDF en `outputs/pdf`. El `output-dir` pertenece a la configuración específica de cada perfil.

### 77.3 Alternativas consideradas

- Un único directorio de salida para todos los formatos.
- Configurar el directorio común en `_quarto.yml`.

### 77.4 Consecuencias

Cada formato puede limpiarse, publicarse o inspeccionarse de manera independiente.

## **78 Usar scripts R reutilizables para preview y render**

### **78.1 Contexto**

Repetir comandos largos de Quarto en terminal dispersa conocimiento operativo y facilita variaciones entre ejecuciones.

### **78.2 Decisión**

IASI utilizará scripts R reutilizables para las operaciones recurrentes de preview y render, como mínimo para HTML y PDF, en lugar de depender únicamente de comandos manuales recordados por el usuario.

### **78.3 Alternativas consideradas**

- Ejecutar siempre comandos Quarto manualmente.
- Ocultar todo el proceso dentro del IDE sin scripts versionados.

### **78.4 Consecuencias**

Las operaciones frecuentes quedan documentadas como código y pueden evolucionar junto al proyecto.

## 79 Separar herramientas principales de trabajo y laboratorios locales

### 79.1 Contexto

El entorno de trabajo puede aprovechar servicios como Codex y Copilot sin renunciar a experimentar con modelos locales. Obligar a todos los componentes a usar el mismo backend reduciría el valor del laboratorio.

### 79.2 Decisión

Codex y Copilot se utilizarán como herramientas principales del workspace cuando resulten adecuadas. Ollama se mantendrá para experimentación local y laboratorios, incluyendo el desarrollo exploratorio de *iasi-graphics*.

### 79.3 Alternativas consideradas

- Estandarizar todo IASI sobre un único proveedor o runtime de modelos.
- Excluir modelos locales del entorno por no ser la herramienta principal.

### 79.4 Consecuencias

Los experimentos pueden evaluar capacidades locales sin condicionar el flujo principal. La metodología y los productos permanecen desacoplados de una única plataforma de IA.

# 80 Postprocesar la salida de Quarto sin forzar su comportamiento

## 80.1 Contexto

Quarto resuelve correctamente su responsabilidad principal: transformar las fuentes del proyecto en salidas HTML, PDF u otros formatos soportados. Sin embargo, algunas necesidades editoriales, estructurales o de publicación de IASI no encajan de forma natural en su modelo de configuración o en su ciclo de renderizado.

Intentar resolver esas necesidades forzando `_quarto.yml`, perfiles, Pandoc, filtros, hooks u otros mecanismos internos aumenta el acoplamiento con Quarto y convierte cada excepción en una negociación con la herramienta.

IASI no necesita que Quarto conozca todas sus reglas. Necesita una salida correcta de Quarto sobre la que pueda aplicar después sus propias transformaciones.

## 80.2 Decisión

IASI utilizará Quarto dentro de los límites naturales de su responsabilidad y aplicará, cuando sea necesario, una fase explícita de **postprocesado** sobre la salida generada.

El flujo conceptual será:

```
source
↓
Quarto
↓
output de Quarto
↓
postprocesado IASI
↓
validación
↓
artefacto final
```

Quarto realizará el renderizado. IASI actuará después sobre el resultado para resolver necesidades que no corresponda implementar dentro de Quarto.

El postprocesado podrá incluir, entre otras operaciones, ajustes editoriales, transformación o reorganización de recursos, incorporación de metadatos, empaquetado y preparación de artefactos para publicación.

No se modificará ni forzará el comportamiento interno de Quarto cuando la misma necesidad pueda resolverse de forma más simple y desacoplada después del renderizado.

### 80.3 Alternativas consideradas

- Resolver todas las necesidades mediante configuración nativa de Quarto.
- Introducir filtros, hooks o extensiones para cada ajuste requerido por IASI.
- Modificar la salida fuente antes del render para inducir el resultado deseado.
- Mantener lógica específica de publicación mezclada con la configuración de Quarto.

### 80.4 Consecuencias

Quarto queda tratado como un motor de renderizado con una responsabilidad clara, mientras IASI conserva el control de su propio proceso de publicación.

Las particularidades de IASI pueden evolucionar sin depender de detalles internos de Quarto ni aumentar innecesariamente la complejidad de su configuración.

La salida directa de Quarto pasa a ser un artefacto intermedio cuando exista postprocesado. El artefacto final es el resultado posterior a las transformaciones y validaciones de IASI.

Esta decisión establece además un criterio general de ingeniería: **no luchar contra una herramienta para obligarla a resolver aquello que queda fuera de su responsabilidad natural; utilizarla para lo que hace bien y continuar el proceso fuera de ella.**

**Part X**

**Pendiente de clasificar**

Esta sección reúne los Engineering Decision Records clasificados en el área **Pendiente de clasificar** (99).

Actualmente no hay EDR registrados en esta área.